
SunSpot Protocol

Proposal A

Author	retc3, SunSpot maintainer
Document	SSP-PROP-A, version 1.0.0
Date	1 August 2026
Protocol version	SSP/1

Change control	SunSpot maintainers
-----------------------	---------------------

Section 1.3 states what is fixed, what is open, and what changes before this text becomes the interoperability reference for SSP/1.

Abstract

The SunSpot Protocol (SSP) carries private messages between people without a central operator. It gives end-to-end confidentiality, forward secrecy, post-compromise security, and strong resistance to metadata collection.

SSP has no directory, no phone numbers, and no accounts. An identity is a self-authenticating, hash-linked, append-only log that holds device keys, rotations, revocations, and recovery rules. The identifier of an identity is the hash of the first entry of that log. It is stable for the life of the identity, and it survives full key replacement.

The identity of a user and the address that the network uses are separate. A sender and a recipient derive an opaque 32-byte mailbox address for their pair, and that address changes every day. A relay stores ciphertext under mailbox addresses. It never learns an identity, a display name, a social graph, or a group membership.

Relays are store-and-forward nodes. They are interchangeable and untrusted. A relay accepts a deposit only when the recipient has registered a deposit key for that mailbox. That one rule removes open relaying, removes the need for a reputation system, and confines proof of work to first contact.

Transport uses QUIC with raw public keys, or TCP with a Noise IK handshake where a client must cross a censored or onion network. Session key agreement combines X25519 with ML-KEM-768. Message keys come from a double ratchet. Groups use TreeKEM epochs with a deterministic rule for concurrent commits, and need no ordering server. Hashing, key derivation, and content addressing use BLAKE3, and attachments use its tree structure for verified streaming.

Every normative statement carries an identifier, a rationale, and a verification method. Section 4 gives the verification plan. Appendix A gives the traceability matrix.

Prior art

SSP reuses published work and names it where it does.

- The Noise Protocol Framework supplies the handshake patterns and the naming discipline.
- WireGuard shows that a small, opinionated cryptographic core with no agility is a strength.
- The Signal protocol supplies the double ratchet and asynchronous key agreement.
- RFC 9420 supplies TreeKEM.
- SimpleX shows that queue-based addressing removes long-lived network identifiers.
- Briar shows what a protocol looks like when the network is assumed hostile.
- Cabal and Secure Scuttlebutt show the strengths and the costs of append-only logs.
- Tor supplies Equi-X and a decade of measurement against funded denial-of-service attacks.
- Monero research sets the standard for honest statements about the limits of a privacy system.

Where SSP differs from these systems, Section 3 and Appendix F state the difference and defend it.

Contents

Abstract	ii
Prior art	ii
1 Introduction	1
1.1 Purpose	1
1.2 Scope	1
1.3 Status of this document	1
1.4 Product overview	1
1.5 Motivation	4
1.6 Design philosophy	4
1.7 Terminology	5
1.8 Conventions	6
2 References	7
2.1 Normative references	7
2.2 Informative references	8
3 Requirements	8
3.1 System context and stakeholders	8
3.2 Threat model	8
3.3 Design goals	12
3.4 Non-goals	13
3.5 Protocol architecture	13
3.6 Identity, devices, and key management	20
3.7 Cryptographic design	30
3.8 Networking, transport, and relays	37
3.9 Messaging	44
3.10 Abuse prevention	55
3.11 Metadata resistance	59
3.12 Performance	63
3.13 Usability	63
3.14 Interfaces	64
3.15 Design constraints	65
3.16 Quality attributes	65
3.17 Extensibility, versioning, and compatibility	67

4	Verification	69
4.1	Strategy	69
4.2	Selected cases	70
4.3	Parser verification	70
4.4	Property verification	71
4.5	Interoperability suite	71
4.6	Traceability	71
5	Implementation guidance	71
5.1	Shape	71
5.2	Crate boundaries	72
5.3	Zero-copy parsing	72
5.4	Type discipline	72
5.5	Zeroization	73
5.6	Async design	73
5.7	Constant time	73
5.8	Build order	73
6	Research agenda	74
6.1	Private retrieval	74
6.2	Group fan-out	74
6.3	Post-quantum authenticity at an acceptable size	74
6.4	Anonymous credentials for first contact	74
6.5	Traffic analysis at an acceptable cost	74
6.6	Formal verification of the composition	75
A	Traceability matrix	75
B	Constants and registries	77
B.1	Sizes	77
B.2	Times	77
B.3	Argon2id parameters	77
B.4	Derivation labels	77
B.5	Poll interval classes	77
C	Code point registries	79
C.1	Error codes	79
C.2	Message content types	79

D	Object layouts	79
D.1	Mailbox registration	79
D.2	Identity log entry, device addition	79
D.3	Message header	79
E	Test vectors	82
F	Comparison with existing systems	82
G	Acronyms	84

1 Introduction

1.1 Purpose

This specification defines the SunSpot Protocol: the rules that let independent programs exchange private messages with no central operator.

The specification is the source of truth. A team that has this text and the referenced cryptographic standards can write a client that interoperates with every other client, and does not need to read anyone else's source code.

1.2 Scope

In scope:

- Creation, rotation, revocation, and recovery of cryptographic identities.
- Enrolment and removal of the devices that belong to one identity.
- Authenticated, confidential links between a client and a relay.
- Asynchronous delivery between two devices, inside a closed group, and on a one-to-many channel.
- Transfer of attachments with incremental integrity checks.
- Synchronisation of state between the devices of one identity.
- Storage and expiry of ciphertext at a relay.
- Prevention of spam and denial of service with no identity registry.
- Reduction of the metadata that a relay or a network observer can collect.
- Negotiation of versions and of optional capabilities.

Out of scope:

- The user interface, the notification style, and the local database schema.
- Real-time voice and video media transport. Section 3.17 reserves the code points that a later media specification needs.
- Moderation policy, keyword filters, and reporting workflows.
- Interoperation with XMPP, Matrix, SMTP, or SMS.
- Payment, tokens, ledgers, and any mechanism that needs global state.

1.3 Status of this document

SSP/1 is a working draft. The architecture below is complete enough to implement, and three decisions are open for maintainer debate before the wire format is frozen: the group fan-out cost of 3.9.6, the plaintext `ETYPE` field of 3.11.3, and the choice to keep Ed25519 while the key agreement moves to a post-quantum hybrid (3.7.12).

SSP/1 is not compatible with any SunSpot build that came before this document. Section 3.17.3 states why the break happens once.

1.4 Product overview

1.4.1 Position in the system

Today the SunSpot application defines behaviour and the wire format is a side effect of the source code. That order is wrong, and it blocks a second implementation. The target

relationship is strict, and it is shown in Figure 1.

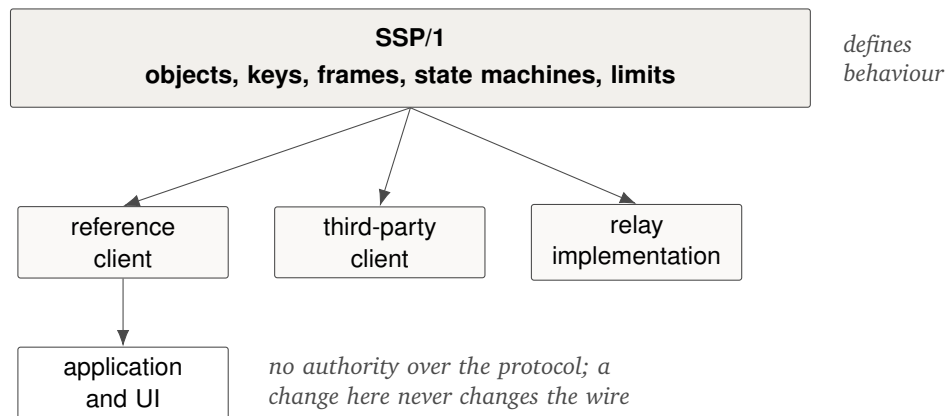


Figure 1. Authority runs one way. The specification constrains every implementation, and no implementation constrains the specification.

SSP-GEN-001 An implementation MUST NOT depend on behaviour that this specification does not state.

Rationale. Undocumented behaviour becomes a de-facto standard and turns one client into the specification.

Verification. Inspection, and the interoperability suite of 4.5.

SSP-GEN-002 The reference client MUST NOT hold any protocol privilege, and MUST NOT use a code point outside the public registry of Appendix C.

Rationale. A privileged client is a central authority in a decentralised network.

Verification. Inspection.

1.4.2 Actors and objects

Actor	Definition
Identity	One long-lived cryptographic principal. Not a person.
Device	One installation that holds private keys for one identity.
Client	The program on a device that speaks SSP.
Relay	A store-and-forward node holding ciphertext under mailbox addresses.
Observer	Any party that watches the network. Not an actor of the system.

Table 1. Actors.

Object	Purpose
Identity log	The append-only, hash-linked record of an identity.
Contact code	The out-of-band token that starts a first contact.
Mailbox	An opaque, rotating destination at a relay.
Envelope	The unit that a relay stores and forwards.
Message	The end-to-end object inside an envelope.
Blob	A content-addressed attachment body.

Table 2. Object classes.

1.4.3 User classes

Ordinary users accept defaults, do not read this document, and scan a code to add a contact. They must get strong security with no configuration. *Operators* run relays; they have a server and modest operational skill, and they must not be able to break confidentiality even if they try. *High-risk users* face a state adversary and will accept extra cost: onion routing, constant-rate cover traffic, and manual key checks.

SSP-USE-001 The protocol **MUST** give ordinary users end-to-end encryption, forward secrecy, and post-compromise security with no user action.

Rationale. Security that needs configuration is security that most users do not have.

Verification. Analysis of the default parameter set in Appendix B.

1.4.4 Limitations

The protocol cannot remove these facts. Section 3.11 gives the detail.

- A relay learns the IP address of the client that connects to it, unless the client uses an onion network or a VPN.
- An observer that watches every link can correlate traffic by timing and by volume. SSP raises the cost of this attack; it does not stop it.
- A device that an adversary controls can copy every plaintext that it reads.
- A member of a group can copy and republish the content of that group.
- Availability depends on relays. An adversary that blocks every relay of a user blocks that user.

1.4.5 Assumptions and dependencies

Ref	Assumption
A-01	Each device has a cryptographically secure random number generator.
A-02	Each device has a monotonic clock, and a wall clock accurate to five minutes for most of the time.
A-03	The user can compare a short code out of band for at least one contact.
A-04	At least one reachable relay is honest about availability.
A-05	X25519, Ed25519, ChaCha20-Poly1305, BLAKE3, and Argon2id resist a classical adversary for the life of SSP/1.
A-06	ML-KEM-768 resists a quantum adversary, or the hybrid of 3.7.12 retains classical security if ML-KEM-768 fails.
A-07	The operating system offers a place to store a wrapped key, or the user supplies a passphrase.

Table 3. Assumptions.

SSP-GEN-003 An implementation **MUST** fail closed if assumption A-01 does not hold, and **MUST NOT** fall back to a weaker source of randomness.

Rationale. A silent fallback to a predictable generator destroys every other guarantee.

Verification. Test with a blocked entropy source.

1.5 Motivation

Four problems drive the design.

The application is the protocol. A second implementation cannot exist while the truth lives in the source of one client. Every fix to that client silently changes the network. This blocks review, blocks embedded clients, and ties the network to one language and one repository.

Encryption is not privacy. Signal encrypts content well and holds a phone number for every user. Matrix encrypts content and then replicates room membership, timestamps, and the social graph to every server in the room. Both show that metadata is the harder problem, and that it leaks through the addressing layer rather than through the cipher.

Identity is either fragile or unstable. A system that uses a raw public key as the identifier cannot rotate that key. A system that uses a server account can rotate keys and needs the server. Both models fail a protocol intended to last a decade, because keys leak and servers close.

Abuse prevention pulls the design toward the centre. Reputation needs a global view. Phone numbers need a registry. Payment needs a ledger. Each solves spam and creates a point that an adversary can attack, buy, or subpoena. A decentralised protocol needs an abuse defence that one untrusted node can enforce from local state alone.

SSP answers these in order: a written protocol, an addressing layer that carries no identity, a log-based identity that rotates keys under a stable name, and a recipient-issued capability that gates every deposit.

1.6 Design philosophy

Protocol first, implementation second, application third. The protocol defines behaviour, the implementation realises it, the application presents it. A layer never defines the behaviour of a layer above it in that list. The practical test: if a change needs an edit to this document it is a protocol change and takes the version rules of 3.17; if it does not, it is a product decision and never touches the wire.

One way to do each thing. Each function has exactly one mandatory mechanism. There is no cipher suite negotiation at the message layer, no choice of serialisation, and no optional integrity mode. WireGuard shows the value of this rule and TLS 1.0 through 1.2 shows the cost of breaking it. A protocol with N optional mechanisms has more than N test paths, and the rare paths hold the defects. Algorithm agility lives at the version boundary, never inside a session.

A relay is furniture. A relay is not a server that a community trusts. It is a disk with a network port: a public locker bank, not a post office. Everything must hold when every relay in the world is hostile, apart from the availability of the messages passing through it.

Refuse to be clever. Any mechanism whose complexity outweighs its gain is rejected, and the rejection is recorded next to the decision it lost to.

Say what you cannot do. Sections 3.2.5, 3.4, and 3.11.6 list the attacks that SSP does not stop. A privacy claim without a stated limit is marketing, and a user who believes a false claim takes a risk they did not choose.

Canonical bytes. Every object that a party signs or hashes has exactly one valid encoding, and a parser rejects anything else. This removes signature malleability, removes hash confusion, and makes content addressing sound.

1.7 Terminology

1.7.1 Technical verbs

These verbs carry these meanings throughout. An implementation description **MUST NOT** substitute a synonym.

Verb	Meaning
encrypt, decrypt	apply, or remove, an authenticated cipher
sign, verify	produce, or check, a digital signature
hash	compute a fixed-length digest of a byte string
derive	compute a key from other key material with a KDF
authenticate	prove the origin and the integrity of data
negotiate	agree a version or a capability set
serialise, parse	write, or read, the wire encoding
publish, subscribe	send to, or register interest at, a relay
rotate	replace a key and keep the identifier
revoke	declare a key invalid from a stated point
delegate	give a capability to another key
ratchet	advance a key chain in one direction only
commit	apply a group state change at an epoch boundary
zeroize	overwrite key material in memory with zero bytes

Table 4. Technical verbs.

1.7.2 Technical names

Name	Definition
Identity	The principal that an identity log defines.
SunSpot identifier (SID)	The 32-byte BLAKE3 hash of the genesis entry of an identity log.
Identity log	The signed, hash-linked, append-only history of an identity.
Genesis entry	Entry 0 of an identity log.
Root key	The Ed25519 key pair that signs identity log entries.
Recovery key	An offline Ed25519 key pair that can replace a lost root key.
Device	One installation that holds keys for one identity.
Device key	The Ed25519 signature key of a device.
Link key	The X25519 static key of a device.
Prekey	A one-time X25519 key and a one-time ML-KEM-768 key, used once.
Contact code	A token carrying a SID, a relay set, and an introduction mailbox.
Mailbox	A destination at a relay, with an address and a deposit key.
Mailbox address	A 32-byte opaque identifier of a mailbox.

Name	Definition
Deposit key	The Ed25519 key pair that authorises a deposit into a mailbox.
Envelope	The outer object that a relay stores. It holds ciphertext.
Message	The end-to-end object inside an envelope.
Blob	A content-addressed attachment body, split into 64 KiB chunks.
Relay	A store-and-forward node.
Relay set	The ordered list of relays that serve one device.
Session	The double ratchet state between two devices.
Group	A closed set of identities that share a TreeKEM tree.
Epoch	The interval between two group commits.
Channel	A one-to-many stream with one publisher.
Wire encoding (SWE)	The canonical binary encoding of 3.5.4.

Table 5. Technical names.

1.7.3 Security terms

Forward secrecy: an adversary holding current keys cannot read past messages. *Post-compromise security*: once an adversary loses access, the parties heal the session and the adversary loses the ability to read new messages. *Deniability*: a recipient cannot prove to a third party that a given sender wrote a given message. *Unlinkability*: an observer cannot tell that two events relate to the same principal. *Fork*: two valid identity log entries that share a parent; always a security event.

1.8 Conventions

1.8.1 Normative keywords

MUST, MUST NOT, REQUIRED, SHOULD, SHOULD NOT, RECOMMENDED, MAY, and OPTIONAL carry the meanings of RFC 2119 and RFC 8174, and are normative only where they appear in small capitals, as here.

1.8.2 Requirement identifiers

A requirement identifier has the form `SSP-CAT-NNN`: the protocol prefix, a three-letter category, and a serial number within that category. Identifiers are permanent. A withdrawn requirement keeps its identifier and gains the text *Withdrawn*, so that the traceability matrix stays stable across revisions. Each requirement holds exactly one testable statement; a requirement holding two is a defect and a reviewer should report it.

1.8.3 Verification methods

Inspection: a reviewer reads the code, the document, or the object. *Analysis*: a reviewer uses a model, a proof, or a calculation. *Demonstration*: an operator runs the system and observes the behaviour. *Test*: an automated case gives a pass or a fail against a known vector.

1.8.4 Notation

All integers on the wire are unsigned and big-endian; there are no variable-length integers. `u8` to `u64` denote unsigned integers of that width, and $[N]$ denotes a fixed field of N bytes.

Code	Category	Code	Category
GEN	General and governance	MDR	Metadata resistance
IDN	Identity	SEC	Security attribute
DEV	Device	PRV	Privacy attribute
KEY	Key management	PRF	Performance
CRY	Cryptography	USE	Usability
NET	Networking and transport	IFC	Interface
RLY	Relay	CON	Design constraint
MSG	Message transport	EXT	Extensibility and version
GRP	Group and channel	ATT	Attachment
SYN	Synchronisation	STO	Storage
ABU	Abuse prevention		

Table 6. Requirement categories.

Concatenation is written $\parallel$. $H(x)$ is BLAKE3-256 of x with a 32-byte output, $KH(k, x)$ is BLAKE3 in keyed mode with the 32-byte key k , and $KDF(k, \text{label}, n)$ is BLAKE3 in keyed mode with k over the label bytes with an n -byte extendable output. Time values are $_{\text{u64}}$ milliseconds since the UNIX epoch, in UTC. A byte is eight bits; KiB is 1024 bytes and MiB is 1 048 576 bytes.

2 References

2.1 Normative references

An implementation conforms to every document in Table 7.

Table 7. Normative references.

Tag	Reference
RFC 2119	Bradner, S. <i>Key words for use in RFCs to indicate requirement levels</i> . BCP 14, 1997.
RFC 8174	Leiba, B. <i>Ambiguity of uppercase vs lowercase in RFC 2119 key words</i> . 2017.
RFC 7748	Langley, A., Hamburg, M., Turner, S. <i>Elliptic curves for security</i> . 2016. X25519.
RFC 8032	Josefsson, S., Liusvaara, I. <i>Edwards-curve digital signature algorithm</i> . 2017. Ed25519.
RFC 8439	Nir, Y., Langley, A. <i>ChaCha20 and Poly1305 for IETF protocols</i> . 2018.
RFC 9106	Biryukov, A., et al. <i>Argon2 memory-hard function</i> . 2021.
RFC 9000	Iyengar, J., Thomson, M. <i>QUIC: a UDP-based multiplexed and secure transport</i> . 2021.
RFC 9001	Thomson, M., Turner, S. <i>Using TLS to secure QUIC</i> . 2021.
RFC 9221	Pauly, T., Kinnear, E., Schinazi, D. <i>An unreliable datagram extension to QUIC</i> . 2022.
RFC 7250	Wouters, P., et al. <i>Using raw public keys in TLS and DTLS</i> . 2014.
RFC 9420	Barnes, R., et al. <i>The messaging layer security protocol</i> . 2023. TreeKEM only.
FIPS 203	NIST. <i>Module-lattice-based key-encapsulation mechanism standard</i> . 2024. ML-KEM-768.

Tag	Reference
BLAKE3	O'Connor, J., Aumasson, J.-P., Neves, S., Wilcox-O'Hearn, Z. <i>BLAKE3: one function, fast everywhere</i> . 2020.
Noise	Perrin, T. <i>The Noise protocol framework</i> , revision 34. 2018.
BIP-350	<i>Bech32m format for v1+ witness addresses</i> . 2021.
Equi-X	The Tor Project. <i>Equi-X proof-of-work scheme</i> , Tor proposal 327. 2023.
Bao	O'Connor, J. <i>Bao: verified streaming for BLAKE3</i> . 2020.
ISO 29148	ISO/IEC/IEEE 29148:2018. <i>Requirements engineering</i> .
ASD-STE100	<i>Simplified Technical English specification</i> , issue 8. 2021.

2.2 Informative references

Table 8. Informative references.

Tag	Reference
Double Ratchet	Marlinspike, M., Perrin, T. <i>The double ratchet algorithm</i> . Signal, 2016.
X3DH	Marlinspike, M., Perrin, T. <i>The X3DH key agreement protocol</i> . Signal, 2016.
PQXDH	Kret, E., Schmidt, R. <i>The PQXDH key agreement protocol</i> . Signal, 2023.
Sealed sender	Signal. <i>Sealed sender</i> . 2018.
WireGuard	Donenfeld, J. <i>WireGuard: next generation kernel network tunnel</i> . NDSS, 2017.
SimpleX	SimpleX Chat. <i>SimpleX messaging protocol</i> . 2023.
Briar	Briar Project. <i>Bramble transport protocol and design documents</i> . 2020.
Cabal	Cabal Club. <i>Cabal protocol specification</i> . 2020.
SSB	Tarr, D., et al. <i>Secure Scuttlebutt protocol guide</i> . 2019.
Session	Oxen. <i>Session protocol whitepaper</i> . 2020.
Matrix	The Matrix.org Foundation. <i>Matrix specification</i> . 2024.
Tor	Dingledine, R., Mathewson, N., Syverson, P. <i>Tor: the second-generation onion router</i> . 2004.
Monero	Monero Research Lab. <i>MRL bulletins and technical notes</i> . 2014–2024.
Loopix	Piotrowska, A., et al. <i>The Loopix anonymity system</i> . USENIX Security, 2017.
ZIP 215	Hopwood, D. <i>Explicitly defining and modifying Ed25519 validation rules</i> . 2020.
RFC 9180	Barnes, R., et al. <i>Hybrid public key encryption</i> . 2022.

3 Requirements

3.1 System context and stakeholders

Figure 2 shows the parties and the one deliberate absence: there is no relay-to-relay protocol. Section 3.8.9 defends that.

3.2 Threat model

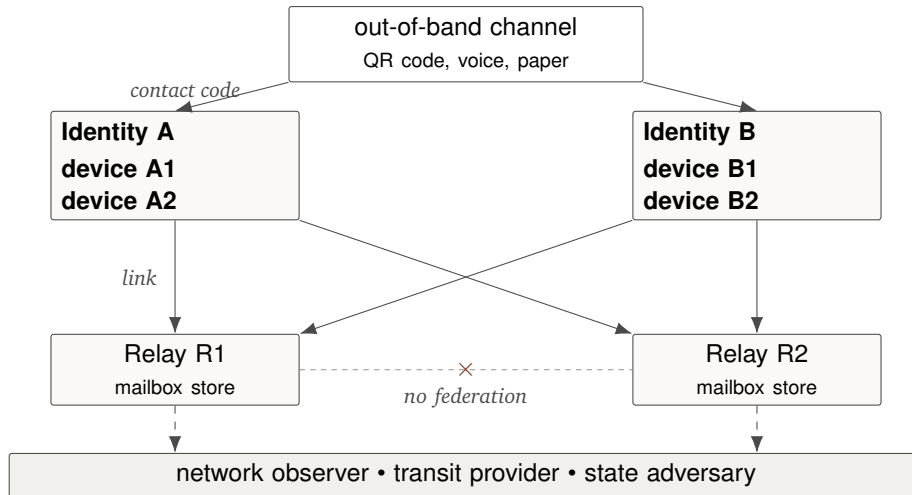


Figure 2. System context.

Stakeholder	Primary concern	Answered in
User at risk	Confidentiality, metadata, deniability	3.7, 3.11
Ordinary user	Reliable delivery, multi-device, low battery cost	3.9, 3.12
Relay operator	Bounded cost, no liability, simple operation	3.8.7, 3.10
Implementer	Complete rules and test vectors	3.5, 4
Maintainer	Long-term stability, safe extension	3.17
Reviewer	Explicit threat model and stated limits	3.2, 3.11.6

Table 9. Stakeholder concerns.

3.2.1 Assets

The protocol protects these assets in this order of priority:

1. Message content.
2. The social graph: the set of pairs that communicate.
3. Group membership.
4. The activity pattern of a user over time.
5. The long-term private keys of an identity.
6. The availability of delivery.

The order is operative. Where a design choice trades availability against content or the social graph, content and the social graph win and availability is allowed to suffer. Section 3.10 applies this rule to spam defence.

3.2.2 Adversary classes

Table 10. Adversary classes.

Ref	Adversary	Capability
T1	Passive local observer	Reads packets on one link: addresses, times, sizes.
T2	Active local observer	Drops, delays, injects, and replays on one link.
T3	Honest-but-curious relay	Reads all state it stores. Follows the protocol.

Ref	Adversary	Capability
T4	Malicious relay	Drops, reorders, replays, lies, serves a forked view.
T5	Relay coalition	Shares logs across every relay it runs.
T6	Global passive adversary	Observes every link at the same time.
T7	Compromised device	Holds current keys and plaintext of one device.
T8	Malicious contact	An authorised peer that copies and republishes.
T9	Malicious group member	Inside a group; sees membership and content.
T10	Sybil operator	Creates unlimited identities and relays at low cost.
T11	Botnet	10^5 to 10^7 hosts with distinct addresses.
T12	State adversary	T2, T5, T6, plus legal power and border filtering.
T13	Supply-chain adversary	Ships a modified client to a subset of users.
T14	Future quantum adversary	Records traffic now, breaks X25519 later.

3.2.3 What each adversary obtains

Asset	T1	T3	T4	T5	T6	T7	T9	T12
Message content	–	–	–	–	–	yes	yes	–
Social graph	partial	–	–	partial	partial	yes	–	partial
Group membership	–	–	–	–	–	yes	yes	–
Activity pattern	yes	yes	yes	yes	yes	yes	partial	yes
Long-term keys	–	–	–	–	–	yes	–	–
Availability	delay	yes	yes	yes	–	yes	–	yes

Table 11. Adversary capability against SSP/1 as specified. T7 and T9 obtain content only for the device or the group they are inside.

The partial results need explanation. T1 sees that a client talks to a relay, and not which mailboxes it reads, because the mailbox list travels inside the encrypted link. T5 sees that one client subscribes to a set of mailbox addresses across the relays it runs; 3.11.4 bounds that with a sharding rule. T6 correlates a send at one client with a fetch at another, which is a real attack that SSP does not stop. T12 can block SSP at a border, and 3.8.3 raises the cost without guaranteeing reachability.

3.2.4 Threats in scope

Table 12. Threats the protocol defends against.

Ref	Threat	Defence
TH-01	Read of content by a relay or an observer	3.7, 3.9.1
TH-02	Retrospective decryption after a key leak	3.9.2
TH-03	Retrospective decryption by a quantum adversary	3.7.12
TH-04	Impersonation of an identity	3.6.2
TH-05	Silent addition of a device by an attacker	3.6.6
TH-06	Silent recovery takeover with a stolen recovery key	3.6.10

Ref	Threat	Defence
TH-07	Replay of an envelope	3.5.6
TH-08	Reorder or drop by a relay	3.9.4
TH-09	A relay serves a forked identity log	3.6.11
TH-10	Enumeration of users	3.8.8
TH-11	Correlation of a mailbox address to an identity	3.8.6
TH-12	Size and count analysis at a relay	3.11.2
TH-13	Spam to an unknown recipient	3.10.2
TH-14	Flood of a known mailbox	3.10.1
TH-15	Resource exhaustion of a relay	3.10.3
TH-16	Sybil creation of identities to gain trust	3.10.4
TH-17	Group membership leak to a relay	3.9.6
TH-18	A removed member reads new messages	3.9.6
TH-19	A parsing defect that a hostile object triggers	3.5.4
TH-20	Downgrade of the negotiated version set	3.17.2

3.2.5 Threats out of scope

This list is normative. An implementation **MUST NOT** claim a defence that it excludes.

Table 13. Threats the protocol does not address.

Ref	Threat	Reason
TX-01	Traffic confirmation by a global passive adversary	A defence needs constant-rate cover traffic on every device at all times. The battery and bandwidth cost make that unusable, and a partial defence gives false confidence. 3.11.5 offers an opt-in mode.
TX-02	Plaintext capture on a device an adversary controls	No protocol can stop an endpoint reading its own plaintext.
TX-03	Republication by an authorised recipient	The recipient must be able to read the content.
TX-04	Coercion of a user to unlock a device	The protocol offers crypto-erase of old keys and nothing more. Duress modes make a promise they cannot keep.
TX-05	A modified client the user installed	Reproducible builds and signing are a distribution problem.
TX-06	A state that blocks all internet access	Beyond the reach of any protocol.
TX-07	Fingerprinting by local clock, fonts, or CPU timing	A client fingerprint problem. 3.11.4 covers the protocol-level rules only.
TX-08	Analysis of the size of a very large attachment	Chunks are padded, and the chunk count still leaks an approximate size.
TX-09	Anonymity of the sender towards the recipient	SSP authenticates senders to recipients deliberately. Anonymous messages are a spam vector.

Ref	Threat	Reason
TX-10	Proof that a message was not delivered	Non-repudiation of delivery needs a trusted third party.

3.2.6 Trust assumptions

SSP-SEC-001 A client MUST treat every relay as a member of adversary class T4.

Rationale. A design that assumes an honest relay fails the moment an operator changes or a host provider seizes a disk.

Verification. Analysis of every relay operation in 3.8.4 against T4.

3.3 Design goals

Table 15. Design goals.

Ref	Goal	Measure of success
G-01	Implementation independence	Two clients written from this text, by teams with no contact, interoperate.
G-02	Confidentiality of content	Content is readable only by the intended devices.
G-03	Forward secrecy and post-compromise security	A key leak exposes a bounded window and the session heals.
G-04	Metadata minimisation at relays	A relay holds no identity, no name, no graph.
G-05	Stable identity with rotatable keys	One identifier survives full key replacement.
G-06	Multi-device with no primary device	Any device can act alone and can add or remove another.
G-07	Decentralisation with no privileged node	Anyone can run a relay; no node has a special role.
G-08	Abuse defence from local state	One relay enforces quotas with no global view and no identity.
G-09	Ten-year stability	Core objects and frames do not change for the life of SSP/1.
G-10	Post-quantum confidentiality	Recorded traffic stays private against a later quantum adversary.
G-11	Bounded relay cost	Storage per mailbox and per epoch has a hard limit.
G-12	Small implementable core	The mandatory core fits in roughly 12 000 lines of Rust with no cipher agility.
G-13	Honest documentation of limits	Every privacy claim carries a stated boundary.

SSP-GEN-004 The protocol MUST reach G-01 without a shared library.

Rationale. A shared library that every client must use is the reference client under another name, and it hides protocol rules in code.

Party	Trusted for	Not trusted for
Relay	Availability of the data it holds	Confidentiality, integrity, ordering, honesty, existence
Contact device	Content it sends, its own key state	The keys of any other identity
Out-of-band channel	Integrity of a contact code at first contact	Confidentiality of that code
Operating system	Process isolation and the key store	Confidentiality of past traffic
Group member	Nothing beyond current-epoch content	Membership honesty, forward silence

Table 14. Trust assumptions.

Verification. Demonstration with two independent implementations, plus the vectors of Appendix E.

3.4 Non-goals

Recorded so that a later revision cannot add them by accident.

Table 16. Non-goals.

Ref	Non-goal	Reason
N-01	Anonymity of the sender to the recipient	Contradicts the spam model of 3.10.
N-02	Global message search across relays	Needs plaintext or a searchable index at the relay.
N-03	Public discoverability of users	Every discovery mechanism is an enumeration mechanism.
N-04	Server-side history for new devices	Needs long-term ciphertext under a long-term key, which breaks forward secrecy.
N-05	Federation between relays	Adds a trust relationship, a routing table, and an amplification vector.
N-06	Consensus or a ledger	Needs global state, and global state is a central point.
N-07	Moderation of content by a relay	A relay cannot read content by design.
N-08	Compatibility with pre-SSP/1 builds	3.17.3.
N-09	Rooms above 1024 members with hidden membership	3.9.8 offers channels for the one-to-many case.
N-10	Cipher suite negotiation inside a session	3.7.1.

3.5 Protocol architecture

3.5.1 Layer model

SSP has six layers plus an identity substrate. Each layer has one duty, and no layer reads the payload of the layer above it.

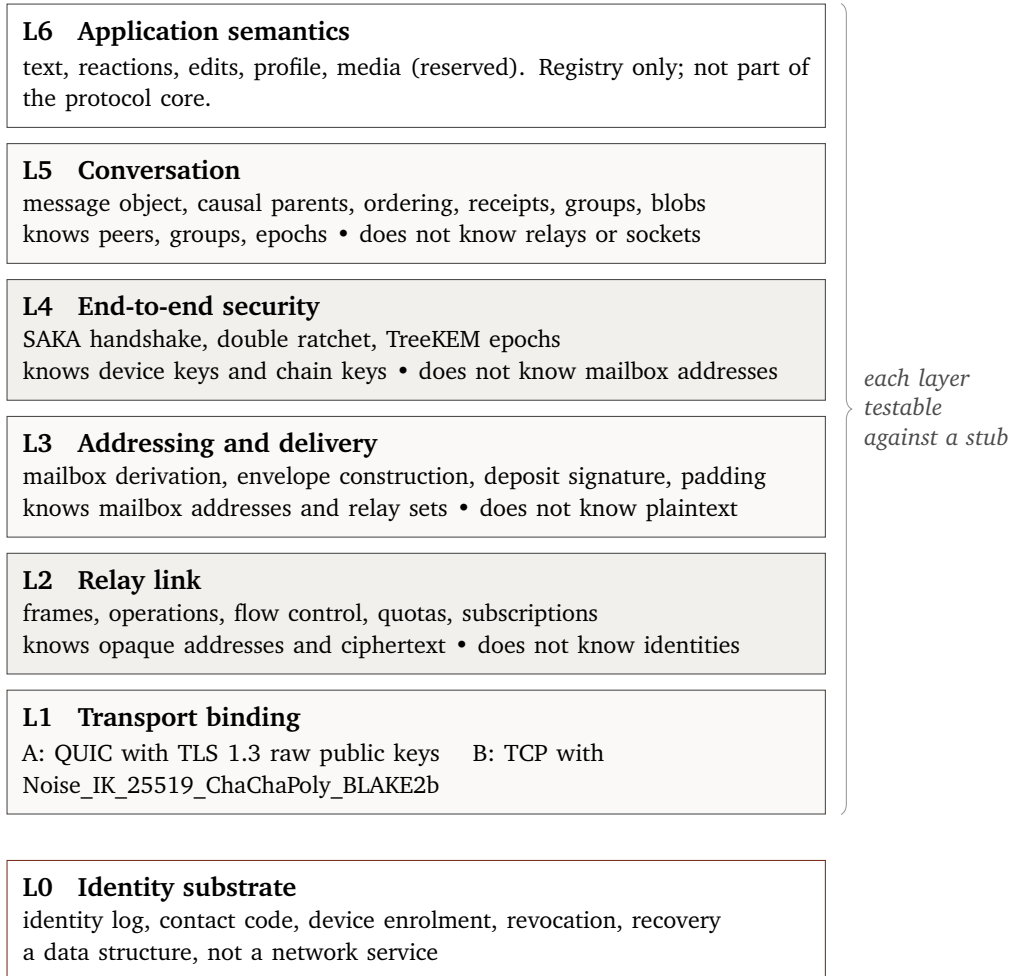


Figure 3. Layer model. L0 sits beneath the stack and beside it: an identity log carries its own proof and can arrive over any channel.

SSP-GEN-005 Each layer **MUST** be implementable and testable with a stub for the layer below.

Rationale. A layer that needs a live socket to run its state machine cannot be tested against vectors, and becomes the place where defects hide.

Verification. Inspection of the crate boundaries in 5.2.

SSP-GEN-006 L2 **MUST NOT** parse, interpret, or act on any byte of the L3 payload beyond the fields that 3.5.5 defines.

Rationale. The mechanical form of “a relay is furniture”.

Verification. Inspection, plus a fuzz campaign feeding random L3 payloads to a relay.

3.5.2 Delivery of one message

Figure 4 traces one text message from device A1 to identity B. Only steps 5 and 9 are visible to the relay, which sees an opaque address, a padded ciphertext, a signature, and a time.

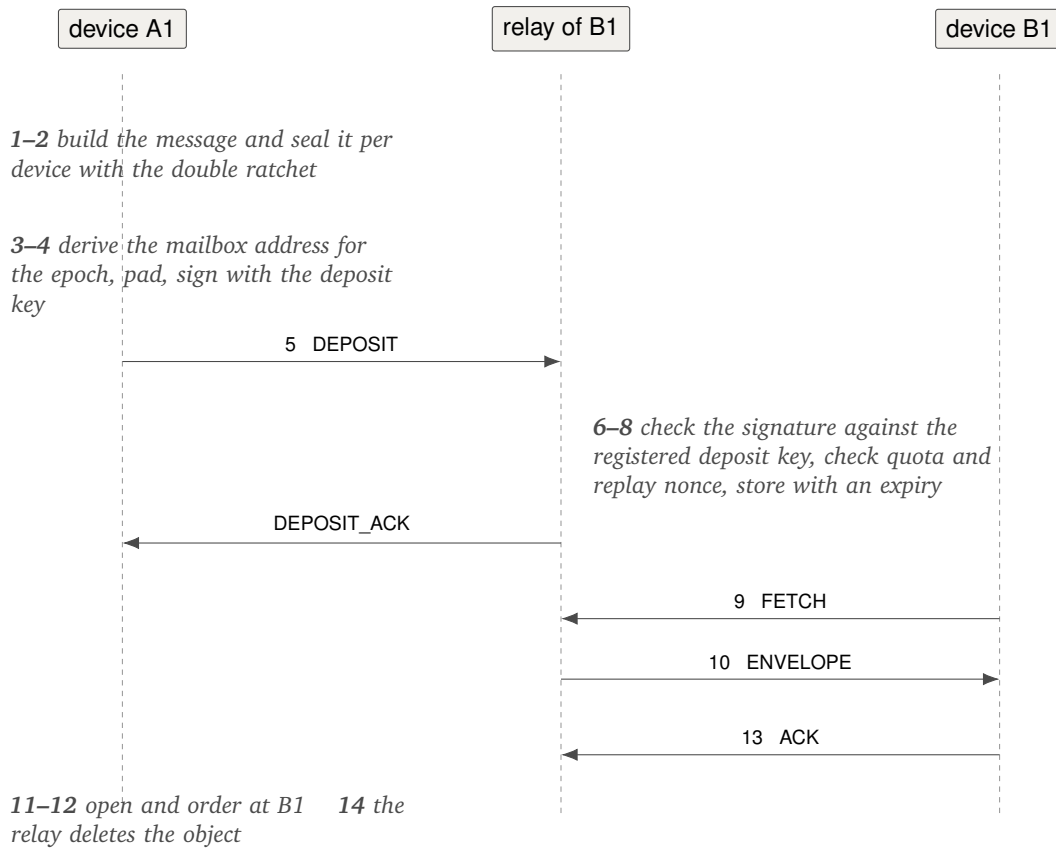


Figure 4. Delivery path for one message.

Delivery to the second device of B repeats steps 3 to 14 with a different mailbox and the relay set of that device; the two mailboxes are unlinkable. Delivery to the second device of the sender takes the same path, through the self group of 3.9.10. There is no special case for a sender that syncs to itself.

3.5.3 Object model

SSP-GEN-007 Every object that a party signs **MUST** begin with a domain separation label, the SSP version, and a context identifier, before any other field.

Rationale. Prevents a signature made in one context from verifying in another. This class of defect has broken several deployed protocols.

Verification. Test with the cross-context signature vectors of Appendix E.

3.5.4 Wire encoding

The SunSpot wire encoding is canonical, binary, and not self-describing. The schema lives in this document; the bytes carry no field names and no type tags except at a declared extension point.

1. Integers are unsigned and big-endian. There are no variable-length integers.
2. A fixed-length field carries no length prefix.
3. A variable-length field carries a `u32` byte-count prefix.
4. A list carries a `u16` count, then the elements in order.
5. There is no padding between fields and no alignment rule.

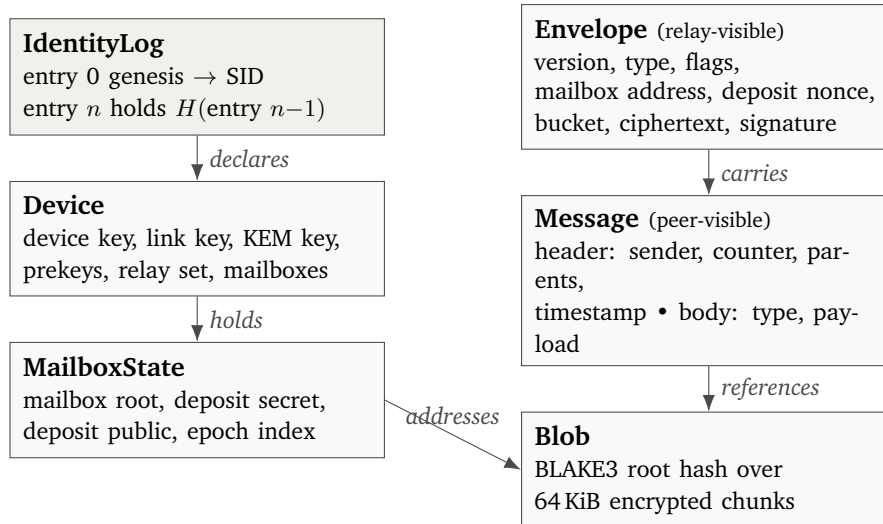


Figure 5. Object model.

6. An optional field is present according to a bit in a flags field. Present optional fields appear in the bit order of that flags field, from the least significant bit upward.
7. An extension is a (u16 type, u32 length, bytes) triple inside an extension list. Bit 15 of the type is the critical bit.
8. A parser rejects any encoding that breaks a rule above, and rejects any trailing byte after the last field.

SSP-CON-001 An implementation **MUST** use this encoding for every byte that crosses the network or that enters a hash or a signature.

Rationale. One encoding for one purpose. A second encoding means two parsers and two attack surfaces.

Verification. Inspection and Test.

SSP-CON-002 A parser **MUST** reject a non-canonical encoding and **MUST NOT** normalise it.

Rationale. A parser that repairs input makes the hash of an object depend on the parser, which breaks content addressing.

Verification. Test with the negative vectors of Appendix E.

SSP-CON-003 A parser **MUST** check every length field against the hard limit in Appendix B before it allocates memory.

Rationale. A length field is the classic remote allocation attack.

Verification. Test, plus a fuzz campaign of at least 10^9 executions.

Rejected encodings.

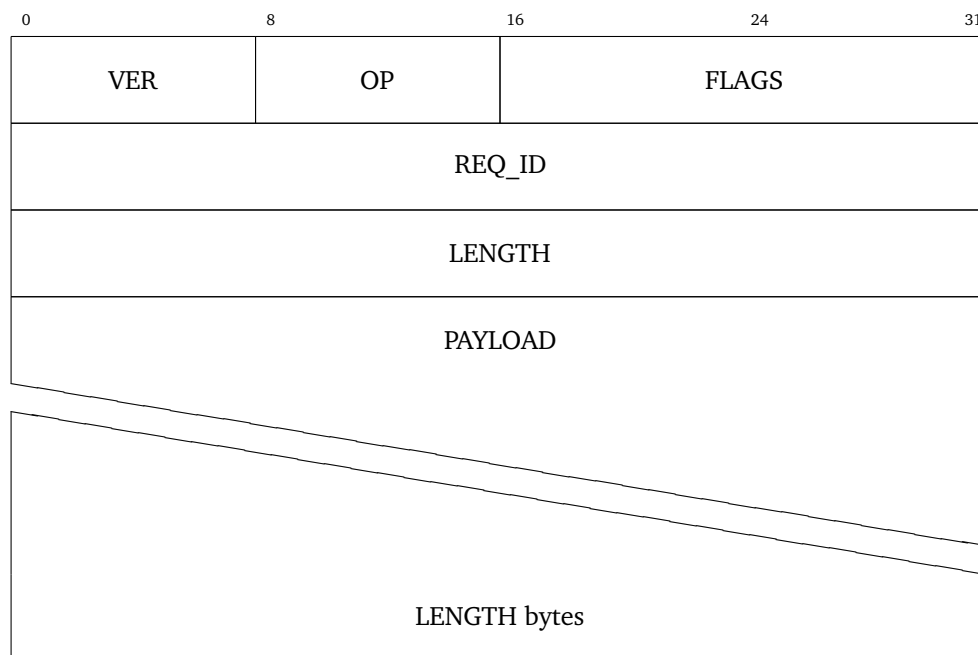
Table 17. Serialisation formats considered and rejected.

Candidate	Reason for rejection
Protocol Buffers	Not canonical. Varints have several valid forms. Unknown fields survive a round trip and change the bytes a signature covers. Presence rules changed between proto2 and proto3, and the ambiguity has produced real security defects elsewhere.
CBOR	The base format is permissive. Canonical CBOR exists, but a parser must reject a great deal of valid CBOR to reach it, and most libraries do not. The self-describing tags add a type-confusion surface SSP does not need.
JSON	Large, no byte type, no canonical number form, costly parser. Text formats invite Unicode normalisation attacks inside signed data.
MessagePack	The same non-canonical problem as CBOR with fewer strict parsers.
bincode, postcard, serde-derived formats	The wire format follows the Rust type layout and the derive macro, which ties the protocol to one language and one library version. A field reorder in the source silently changes the wire.
ASN.1 DER	Canonical, and battle-tested in the negative sense. Parser complexity is very high and the defect history is long.
Cap'n Proto, FlatBuffers	Good zero-copy properties, no canonical form. The pointer layout hands an attacker aliasing tricks inside one buffer.

The cost is a hand-written parser. The return is one valid encoding per object, zero-copy parsing over a borrowed slice, no code generation, and no dependency that can change the wire format in a minor release.

3.5.5 Relay link frame

Every transport binding carries the same frame.

**Figure 6.** L2 frame.

Field	Width	Rule
VER	u8	Frame version, 0x01 for SSP/1. A relay closes the link on an unknown value.
OP	u8	Operation code; registry in Appendix C.
FLAGS	u16	Bit 0 response. Bit 1 more frames follow for this REQ_ID. Bit 2 padding frame, which the receiver discards. Bits 3–15 reserved and zero.
REQ_ID	u32	Chosen by the client and repeated in the response. Zero means no response wanted.
LENGTH	u32	Payload length. At most 1 048 576 bytes for a frame carrying an envelope, 65 536 otherwise.

Table 18. Frame header fields.

SSP-NET-001 A relay **MUST** close the link when a frame declares a LENGTH above the limit for its OP value.

Rationale. The limit has to apply before the relay reads the body, so that one frame cannot exhaust memory.

Verification. Test.

SSP-NET-002 A client and a relay **MUST** accept frames for one REQ_ID out of order with respect to other REQ_ID values.

Rationale. QUIC streams complete in any order, and a strict global order would force the head-of-line blocking that QUIC exists to remove.

Verification. Test with interleaved requests.

Table 19 summarises the operations; Appendix C gives the payload layouts.

Table 19. Relay link operations.

Code	Name	Direction	Purpose
0x01	HELLO	client → relay	Version and capability offer.
0x02	HELLO_ACK	relay → client	Selection, policy, limits.
0x10	MB_REGISTER	client → relay	Create a mailbox with a deposit key and a quota.
0x11	MB_RENEW	client → relay	Extend the life of a mailbox.
0x12	MB_DELETE	client → relay	Delete a mailbox and its content.
0x13	MB_STATUS	client → relay	Report count, bytes, quota state.
0x20	DEPOSIT	sender → relay	Store one envelope.
0x21	DEPOSIT_ACK	relay → sender	Accept, with the assigned sequence.
0x22	DEPOSIT_NAK	relay → sender	Reject, with a reason and any required effort.
0x30	SUBSCRIBE	client → relay	Watch a set of mailboxes.
0x31	UNSUBSCRIBE	client → relay	Stop watching.
0x32	FETCH	client → relay	Read objects after a cursor.
0x33	ENVELOPE	relay → client	Deliver one stored envelope.
0x34	ACK	client → relay	Confirm receipt so the relay can delete.
0x40	BLOB_PUT	client → relay	Store one chunk group.

Code	Name	Direction	Purpose
0x41	BLOB_GET	client → relay	Read one chunk group.
0x50	POW_CHALLENGE	relay → client	Issue an Equi-X challenge.
0x51	POW_SOLUTION	client → relay	Supply the solution.
0x60	PAD	either	A frame carrying only padding.
0x7F	ERROR	either	A terminal error with a code from Appendix C.

3.5.6 Envelope

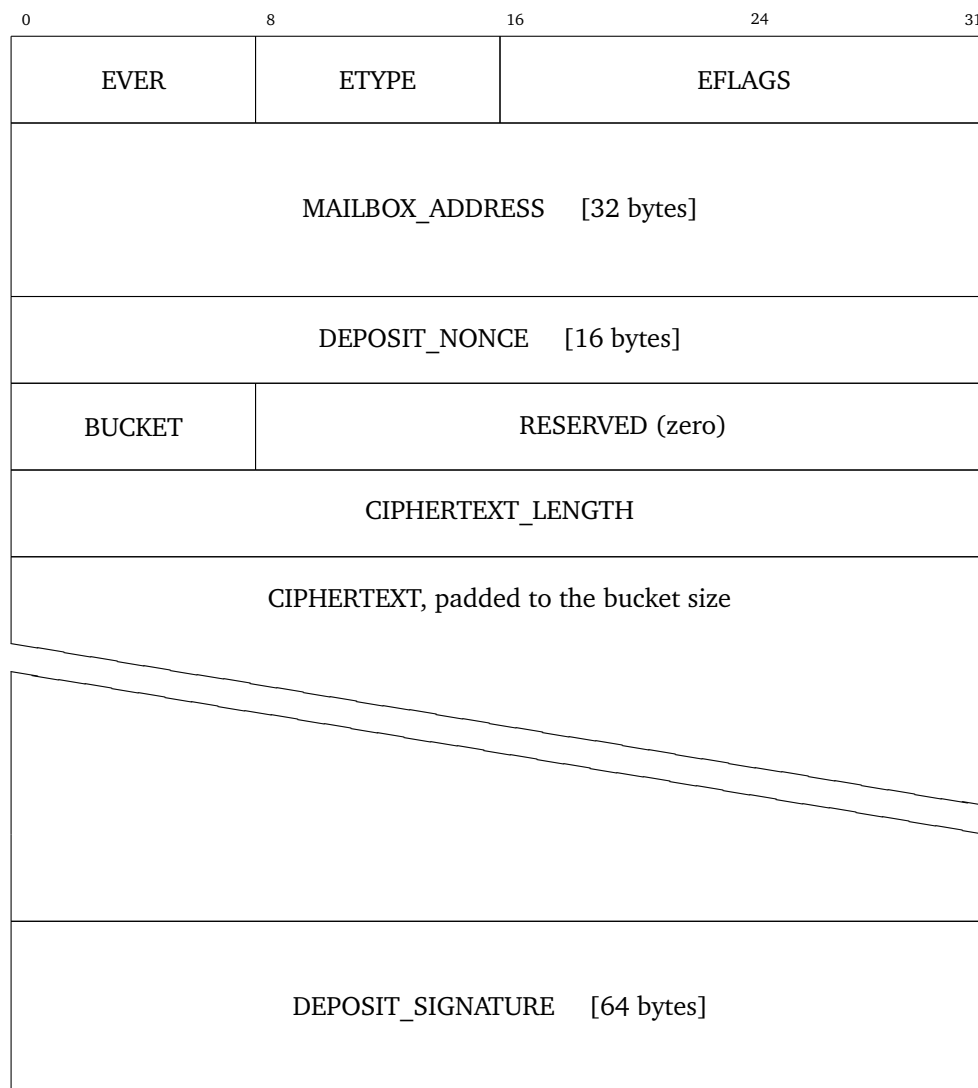


Figure 7. Envelope, the unit a relay stores.

ETYPE is visible to the relay: a deliberate leak of five values, needed for the quota class of 3.10.1 and the expiry class of 3.8.7. Section 3.11.3 states the cost.

SSP-MSG-001 A relay MUST reject a deposit whose signature does not verify against the registered deposit key of the mailbox.

Rationale. This single check carries the whole abuse model of 3.10.

Verification. Test.

Field	Rule
EVER	0x01 for SSP/1.
ETYPE	0x01 message, 0x02 introduction request, 0x03 group commit, 0x04 blob notice, 0x05 receipt only.
EFLAGS	Bit 0 the sender wants an acknowledgement. Bit 1 urgent, and the relay should push it to a subscriber first. Bits 2–15 reserved and zero.
MAILBOX_ADDRESS	The output of 3.8.6.
DEPOSIT_NONCE	Sixteen random bytes, held in the replay cache for the life of the object.
BUCKET	The padding bucket index of 3.11.2.
CIPHERTEXT	The L4 output, padded to the bucket size.
DEPOSIT_SIGNATURE	Ed25519 over SSP1/deposit 0x00 EVER ETYPE MAILBOX_ADDRESS DEPOSIT_NONCE BUCKET H(ciphertext).

Table 20. Envelope fields.

SSP-MSG-002 A relay **MUST** reject a deposit whose nonce is already in the replay cache of that mailbox.

Rationale. Stops replay of ciphertext by an observer and stops quota waste.

Verification. Test.

SSP-MSG-003 The ciphertext length **MUST** equal the size of the bucket that BUCKET names.

Rationale. A mismatch would let a sender signal information in the difference between the declared and the real length.

Verification. Test.

3.6 Identity, devices, and key management

3.6.1 Identity model

An identity is not a name and not a public key. It is an append-only, hash-linked, self-authenticating log. Its identifier is the hash of the first entry of that log:

$$\text{SID} = \text{BLAKE3-256}(\text{SWE}(\text{Entry}_0)) \quad 32 \text{ bytes.}$$

The log holds everything another party needs in order to authenticate the identity: the current root key, the device set, revocations, the recovery set, and relay hints.

Model	Example	Failure
Identity is a raw public key	Session, Briar, SSB	The identifier cannot survive key rotation. A leaked key forces a new identity, and every contact must authenticate again out of band.
Identity is a server account	Signal, Matrix, XMPP	Rotation works and the server becomes the root of trust. The server can add a device.
Identity is a log	SSP	The identifier is stable, rotation works, no server is trusted. The cost: a client must fetch and check a log.

Table 21. Identity models.

The cost of the log model is real: a client stores a log per contact, checks every signature in it, and must detect a fork (3.6.11). It is bounded, because a normal log holds fewer than fifty entries in ten years.

SSP-IDN-001 A SunSpot identifier **MUST** be the BLAKE3-256 hash of the canonical encoding of the genesis entry of its identity log.

Rationale. The identifier binds to the initial state, and the hash chain binds every later state to that identifier.

Verification. Test with the vectors of Appendix E.

SSP-IDN-002 A SunSpot identifier **MUST NOT** change for any reason, including full rotation of every key of the identity.

Rationale. G-05. A stable name is what lets a user keep a contact list after a compromise.

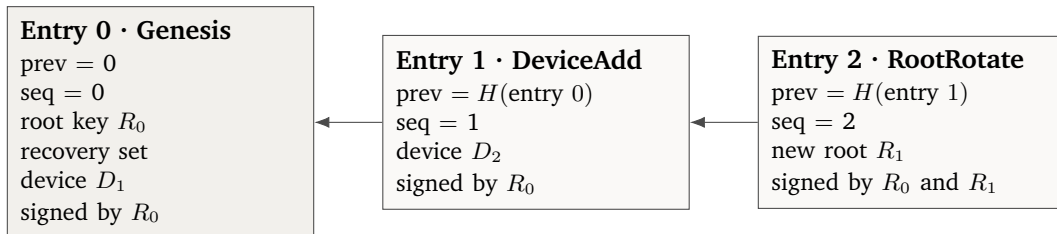
Verification. Analysis, plus the rotation vector.

SSP-IDN-003 The protocol **MUST NOT** carry a human-readable name, a phone number, an email address, or any other external identifier inside an identity log.

Rationale. The log is public to every contact and to every relay that stores a copy. A name in the log is a permanent, non-revocable leak. Display names are per-contact application values, synchronised end-to-end by 3.9.10.

Verification. Inspection of the entry schema.

3.6.2 Log structure



$SID = H(\text{entry } 0)$, fixed for life

Figure 8. Identity log. Each entry names the hash of its predecessor, so the SID commits to the whole history.

Every entry carries the header of Table 22.

SSP-IDN-004 A verifier **MUST** check entries in order from sequence 0, and **MUST** reject the whole log when any entry fails a check.

Rationale. Partial acceptance would let an attacker graft a valid tail onto an invalid trunk.

Verification. Test with the negative vectors of Appendix E.

SSP-IDN-005 A verifier **MUST** reject an entry whose time value is more than 86 400 000 ms after the time value of the entry that follows it.

Rationale. The log is not a clock and the writer is not trusted. The rule catches gross reordering without a timestamp service.

Verification. Test.

Field	Type	Rule
label	[12]	The ASCII bytes <code>SSP1/idlog</code> with two zero bytes.
version	u8	0x01.
entry_type	u8	See Table 24.
seq	u32	Zero for genesis, then plus one per entry.
prev	[32]	Zero for genesis, then the hash of the previous entry.
time	u64	Milliseconds since the UNIX epoch. Advisory.
not_before	u64	When the entry takes effect; used by 3.6.10.
body	var	Type-specific.
signatures	list	One or more (<code>key_id</code> [8], <code>sig</code> [64]) pairs.

Table 22. Entry header.

SSP-IDN-006 A verifier **MUST** reject an entry whose sequence is not exactly one more than its predecessor.
Verification. Test.

Entry type	Required signatures
Genesis	The root key R_0 .
DeviceAdd	The current root key, or a device key holding the enrol capability.
DeviceRemove	The current root key, or any current device key.
RootRotate	The old root key and the new root key.
RecoverySetUpdate	The current root key and a quorum of the current recovery set.
RecoveryClaim	A quorum of the recovery set.
RecoveryVeto	Any current device key.
RelaySetUpdate	Any current device key.
Extension	As the extension registry states.

Table 23. Signature requirements per entry type.

Signature rules. DeviceRemove needs only one device key, deliberately: a user whose device is stolen must be able to remove it from any other device without the root key, which may be offline. The risk is that a compromised device can remove its siblings; 3.6.11 gives the answer, which is a recovery claim.

3.6.3 Entry types

A device record holds the device id (the first eight bytes of $H(\text{device key})$), the Ed25519 device key, the X25519 link key, the 1184-byte ML-KEM-768 key, a capability word, and a timestamp: about 1270 bytes. A log with eight devices is therefore about 10 KiB, which a contact fetches once and afterwards updates by delta.

SSP-IDN-007 A client **MUST** support an identity log of at least 1024 entries and 256 KiB.
Rationale. A smaller limit would reject a valid long-lived identity and split the network.
Verification. Test.

Code	Name	Body
0x01	Genesis	root key, recovery policy, recovery set, first device record
0x02	DeviceAdd	device record
0x03	DeviceRemove	device id, reason, revoke_from
0x04	RootRotate	new root key, reason
0x05	RecoverySetUpdate	recovery policy, recovery set
0x06	RecoveryClaim	new root key, new device record, effect_after
0x07	RecoveryVeto	claim hash
0x08	RelaySetUpdate	device id, relay records
0x09	PrekeyPointer	device id, relay records for the prekey mailbox
0x0A	Close	reason; the identity accepts no further entries
0x0B– 0x7F	Reserved	
0x80– 0xFF	Extension range, critical bit at bit 7	

Table 24. Identity log entry types.

3.6.4 Address format

A user sees a SID as Bech32m text with the human-readable part `sun`:

```
sun1qw508d6qe jxtdg4y5r3zarvary0c5xw7kv8f3t4...
```

SSP-IDN-008 A client **MUST** use Bech32m with the human-readable part `sun` for the text form of a SID.

Rationale. Bech32m detects every error of up to four characters, is case-insensitive, and encodes into the alphanumeric mode of a QR code, which yields a smaller code than base64. Base58 has no checksum design and no error location; hexadecimal is 64 characters with no error detection at all.

Verification. Test with the vectors of Appendix E.

SSP-IDN-009 A client **MUST** reject a SID string with a bad checksum and **MUST NOT** attempt to correct it.

Rationale. Error correction on a security identifier lets an attacker pick a string that corrects into the identifier of the victim.

Verification. Test.

3.6.5 Device model

An identity has one to eight devices. Every device is equal; none is primary and none must be online.

SSP-DEV-001 The protocol **MUST NOT** define a primary device, or a device on which another device depends for its operation.

Rationale. Signal ties a linked device to a phone, so losing the phone breaks the linked devices. A protocol meant to last a decade cannot carry that dependency.

Verification. Inspection of the state machines in 3.6.6, 3.9.6, and 3.9.10.

SSP-DEV-002 An identity **MUST NOT** hold more than eight active devices.

Rationale. A sender encrypts one copy per recipient device, so cost is linear in the device count. In a group of 1024 members, eight devices give a worst case of 8192 copies, the largest fan-out 3.12 accepts. A higher limit would let one member impose unbounded cost on every other member.

Verification. Test with a log declaring nine active devices, which a verifier rejects.

SSP-DEV-003 Every device **MUST** hold its own device key, link key, and KEM key pair, and **MUST NOT** copy a private key to another device.

Rationale. A shared private key removes the ability to revoke one device, and removes the value of the ratchet, because two devices would share one state.

Verification. Inspection and Demonstration during enrolment.

3.6.6 Device enrolment

The new device generates its own keys and the existing device signs a DeviceAdd entry. A short secret travels out of band, so an active network attacker cannot insert a device.

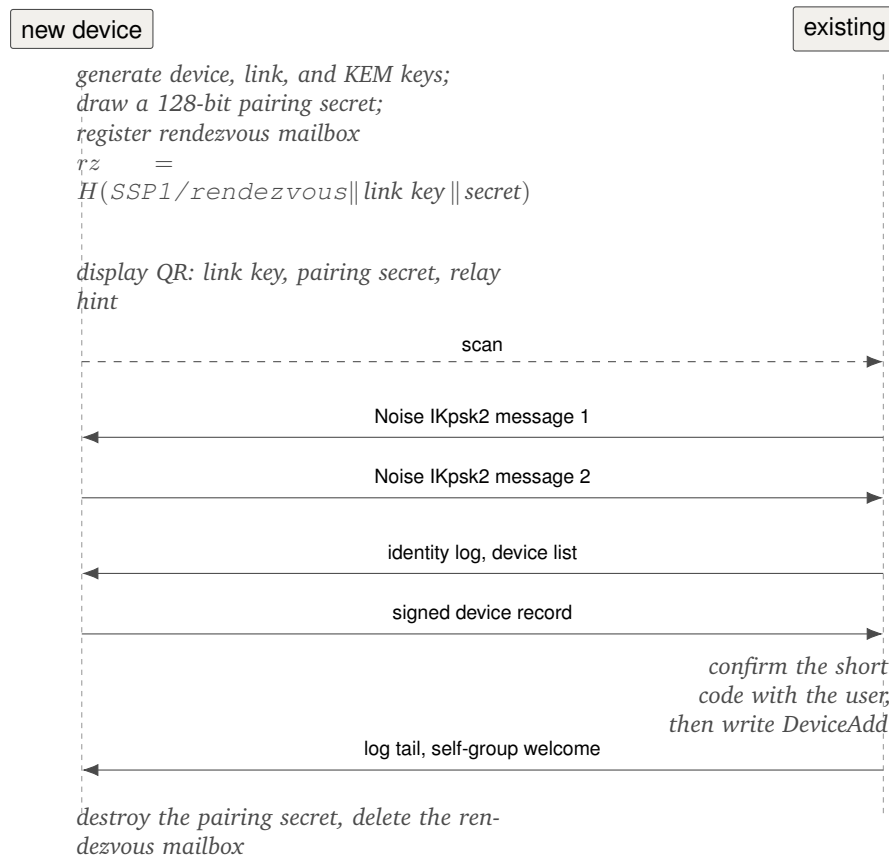


Figure 9. Device enrolment. The initiator is the existing device, which already knows the responder static key because the QR code carries it.

SSP-DEV-004 Enrolment **MUST** use Noise_IKpsk2_25519_ChaChaPoly_BLAKE2b with the responder static key and the pre-shared key taken from the out-of-band channel.

Rationale. IK hides the initiator identity from a passive observer and needs one round trip. The psk2 placement binds the transport keys to the physical scan without changing the security properties of the pattern.

Verification. Test against the Noise vectors and the vectors of Appendix E.

SSP-DEV-005 A client **MUST** show a five-word or eight-digit confirmation code derived from the handshake hash, and **MUST NOT** write the DeviceAdd entry before the user confirms it.

Rationale. The psk defends against a network attacker; the code defends against a user who scanned the wrong thing, and makes the event visible.

Verification. Demonstration.

SSP-DEV-006 A client **MUST** alert the user on every device of the identity when a DeviceAdd entry appears in the log.

Rationale. TH-05. A silent device addition is the classic key-server attack; the log makes it detectable, and detection is worthless if the client hides it.

Verification. Demonstration.

SSP-DEV-007 The pairing secret **MUST** be at least 128 bits, **MUST** be used once, and **MUST** be zeroized after the handshake.

Verification. Inspection and Test.

Matrix mediates verification through the homeserver and Signal relays linked device traffic through its service. SSP keeps the QR model and removes the server from the trust path with the pre-shared key: the rendezvous relay cannot join the handshake, because it does not hold that secret.

3.6.7 Device removal

SSP-DEV-008 A DeviceRemove entry **MUST** carry a `revoke_from` time, and a verifier **MUST** treat every signature by that device at or after that time as invalid.

Rationale. A removal effective from the moment of writing would invalidate the whole history of that device and break the causal record of past messages.

Verification. Test.

SSP-DEV-009 On a DeviceRemove entry, every other device of the identity **MUST** start a commit in the self group and in every group where the identity is a member, within 60 seconds of learning of the entry.

Rationale. Post-compromise security. A removed device holds the current group secret until a commit replaces it.

Verification. Test and Demonstration.

SSP-DEV-010 A peer holding a session with the removed device **MUST** stop that session, delete its state, and reject any message from that device timed at or after `revoke_from`.

Verification. Test.

SSP-DEV-011 A client **MUST** delete the mailbox it holds for a removed peer device, at every relay in the relay set of that device.

Rationale. An orphan mailbox is a place an attacker can leave data, and it consumes a quota.

Verification. Test.

3.6.8 Key hierarchy

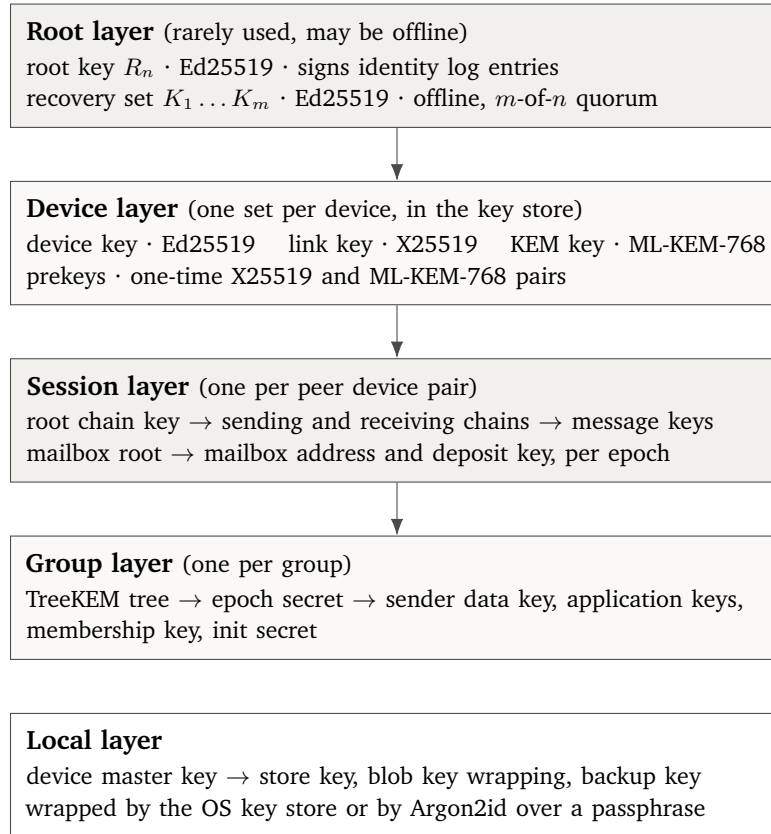


Figure 10. Key hierarchy. Each private key serves exactly one purpose.

SSP-KEY-001 A private key MUST serve exactly one purpose in this hierarchy.

Rationale. Key reuse across a signature scheme and a key agreement scheme has produced real breaks, and separation lets one key rotate without disturbing the others.

Verification. Inspection.

SSP-KEY-002 A client MUST derive every symmetric key as $\text{KDF}(\text{parent}, \text{SSP1} / \dots, \text{length})$, where the label names the exact purpose.

Rationale. Domain separation. Appendix B holds the complete label registry, and an implementation never invents a label.

Verification. Test with the label vectors of Appendix E.

SSP-KEY-003 A client MUST zeroize a key when it leaves the active window, and MUST NOT allow key material to reach a swap file, a core dump, or a log.

Rationale. Forward secrecy at the protocol level is worthless if plaintext keys stay in memory for the life of the process.

Verification. Inspection, plus analysis of the memory rules in 5.5.

3.6.9 Rotation

Key	Trigger	Method
Message key	Every message	Symmetric ratchet
Chain key	Every DH ratchet step	Double ratchet
Session DH key	Every reply, and at most every 100 messages	Double ratchet
Prekey	Every use	One-time; the client uploads a new batch
Link and KEM key	180 days, or suspicion	DeviceAdd of a new record, DeviceRemove of the old
Deposit key	30 days, or abuse	Register a new mailbox, delete the old
Mailbox address	24 hours	Derived from the epoch index; no message needed
Device key	Not rotated	A new device replaces the old one
Root key	365 days, or suspicion	RootRotate entry
Recovery set	User action	RecoverySetUpdate entry

Table 25. Rotation schedule.

SSP-KEY-004 A client **MUST** rotate the root key at least once every 365 days while the identity is active, and **MUST** sign the RootRotate entry with both the old and the new root key.

Rationale. The double signature proves that the holder of the old key intended the change and that the new key exists. A single signature would let an attacker who steals the old key hand control to a key the attacker cannot use.

Verification. Test.

SSP-KEY-005 A client **MUST** keep at least 32 unused one-time prekeys at each relay of its relay set, and **MUST** upload more when the count falls below eight.

Rationale. An empty prekey store forces a fallback to the static link key, which weakens forward secrecy for the first message. An attacker who drains the store on purpose gains that weakness.

Verification. Test with a prekey exhaustion case.

SSP-KEY-006 When the prekey store of a peer is empty, a sender **MUST** use the static link and KEM keys, **MUST** mark the session as a weak start, and **MUST** run a DH ratchet step at the first reply.

Rationale. Refusing to send would turn prekey exhaustion into a denial-of-service attack. The mark and the fast ratchet bound the damage.

Verification. Test.

3.6.10 Recovery

Recovery answers one question: the user lost every device and must keep the SID.

The recovery set holds one to eight Ed25519 public keys and a quorum size m . The default is $m = 1$ with two keys: one derived from a 24-word passphrase with Argon2id and written on paper, and one optional key on a hardware token or held by a trusted person.

SSP-KEY-007 A genesis entry **MUST** carry a recovery set with at least one key.

Rationale. An identity with no recovery path is destroyed by a lost phone, and the user then rebuilds every contact by hand.

Verification. Test.

SSP-KEY-008 A client **MUST** derive a passphrase recovery key with Argon2id, with the parameters of Appendix B, over the salt $H(\text{SSP1/recovery-salt} \parallel \text{normalised passphrase})$ truncated to 16 bytes.

Rationale. A SID-independent salt lets the user recover from the passphrase alone. Argon2id at these parameters costs about one second and 64 MiB, which puts an offline attack on a 24-word passphrase far beyond the value of the target.

Verification. Test with the vectors of Appendix E.

The delay window. A stolen recovery key must not give a silent takeover.

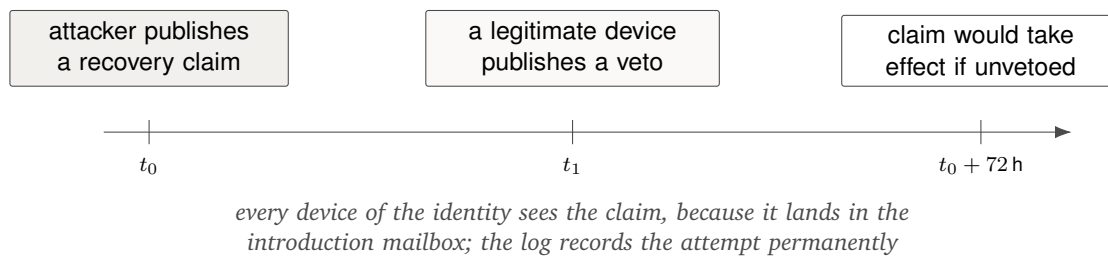


Figure 11. Recovery claim and veto.

SSP-KEY-009 A recovery claim **MUST** carry an `effect_after` value at least 259 200 000 ms (72 hours) after its own time value.

Rationale. The window must be long enough for a user with one offline weekend to notice. Shorter favours the attacker; longer makes real recovery painful.

Verification. Test.

SSP-KEY-010 A verifier **MUST** treat a recovery claim as void when a valid veto for that claim exists with a time before `effect_after`.

Verification. Test.

SSP-KEY-011 A client **MUST** raise a high-priority alert on every device, and show a persistent warning, on seeing a recovery claim for its own identity or for a contact.

Rationale. The delay window has value only when the user learns of the claim.

Verification. Demonstration.

SSP-KEY-012 After a recovery takes effect, a contact client **MUST** mark the contact as having changed keys through recovery, **MUST NOT** drop the contact, and **MUST** ask the user to authenticate the contact again out of band before the next message.

Rationale. Recovery is exactly what an attacker with the recovery key wants. The protocol keeps the SID and moves the trust decision to the user, who can telephone the contact.

Verification. Demonstration.

Alternative	Reason it is not the default
m -of- n social recovery	Leaks the social graph to the participants and needs them online and competent. Available as a policy, not as the default.
Server-side backup with a PIN	Works, and needs trust in an enclave vendor and an operator. SSP has no operator.
Shamir shares across relays	Relays are untrusted and interchangeable; a share at a relay is a share at an attacker.
No recovery at all	A lost device destroys the identity and the user rebuilds every contact by hand.

Table 26. Recovery mechanisms considered.

3.6.11 Fork detection and compromise

Two valid entries sharing a parent form a fork. An honest client never writes one, so a fork means either that an attacker holds a signing key or that a relay serves an inconsistent view.

SSP-IDN-010 A client **MUST** store the hash of the highest entry seen for each contact and compare a new tail against it.

Rationale. The client-side transparency check, at a cost of 32 bytes per contact.

Verification. Test.

SSP-IDN-011 A client **MUST** fetch a changed identity log from at least two relays in the relay set of that contact, and **MUST** report a mismatch as a fork.

Rationale. One relay can serve a split view; two relays must collude, which raises the cost.

Verification. Test with a lying relay in the harness.

SSP-IDN-012 On a fork, a client **MUST** stop all sends to that identity, show both heads to the user, and **MUST NOT** choose a head by itself.

Rationale. Automatic resolution is what the attacker wants. A rule that picks the longer chain lets an attacker win by writing more entries.

Verification. Demonstration.

SSP-IDN-013 A client **MUST** include the known head hash of the identity log of a peer inside its encrypted messages to that peer, at least once every 24 hours.

Rationale. This turns every conversation into a transparency gossip channel at almost no cost. An attacker who forks a log must then keep the fork consistent across every conversation of the victim, which is not achievable in practice.

Verification. Test.

3.6.12 Contact codes

There is no directory. A user reaches a new contact only with a contact code, which carries the SID, the introduction mailbox address, the private seed of the deposit key for that mailbox, up to four relay hints, the log head at the time of issue, and optional single-use and expiry flags. The text form uses Bech32m with the human-readable part `suncode`; the QR form carries the same bytes.

Situation	Response
One device lost, others available	Any other device writes DeviceRemove. Every group commits. Sessions restart.
Every device lost, recovery key available	Recovery claim, 72-hour wait, new root key, new device.
Root key leaked, a device available	Rotate the root key now, alert every contact.
Recovery key leaked	Update the recovery set now, signed by the root key.
The attacker removed the honest devices	The honest user files a recovery claim; the 72-hour window applies to the attacker as well, and the honest user holds the recovery key.
Every key lost, no recovery	The identity is dead. Write a Close entry if any device still works.

Table 27. Response to compromise.

SSP-IDN-014 A contact code **MUST** carry the private seed of the deposit key of the introduction mailbox.

Rationale. The holder needs the ability to deposit there. The key is not a shared secret between two parties; it is a token meaning “the bearer met the owner”. Anyone who copies the code can send one introduction, and 3.10.2 bounds that with proof of work and a per-mailbox quota.

Verification. Inspection and Test.

SSP-IDN-015 A client **MUST** support single-use contact codes, and **MUST** rotate the introduction mailbox when the user requests a new code.

Rationale. A code posted in public becomes a spam target; a single-use code shown to one person does not.

Verification. Test.

SSP-IDN-016 A client **MUST NOT** publish, upload, or index a contact code anywhere in the protocol.

Rationale. N-03. A published set of codes is a user directory.

Verification. Inspection.

SSP-IDN-017 A client **MUST** show that an incoming introduction came from a named existing contact, and **MUST NOT** accept it automatically.

Rationale. An introduction is a transfer of trust, and the user makes it.

Verification. Demonstration.

3.7 Cryptographic design

3.7.1 Selection criteria

Every primitive was judged against these criteria, in order:

1. **Misuse resistance.** How does it fail when an implementer makes an ordinary mistake? A primitive that fails catastrophically on nonce reuse ranks below one that degrades.
2. **Constant-time software implementation.** It must be safe on a device with no hardware

acceleration.

3. **Public analysis.** Years of open review.
4. **Availability.** At least two independent Rust implementations and one audited C implementation.
5. **Portability.** It must run on a 32-bit target with no allocator.
6. **Performance.** Ranked below everything above it.
7. **Size on the wire.** Small output helps padding and battery cost.

SSP-CRY-001 SSP/1 MUST define exactly one algorithm for each cryptographic role, and MUST NOT negotiate an algorithm inside a session.

Rationale. Agility inside a session creates a downgrade surface and doubles the test matrix. It has never saved a deployed protocol from a real break in less time than a version bump would have taken.

Verification. Inspection.

3.7.2 Primitive set

Role	Algorithm	Size
Hash, content address	BLAKE3-256	32 B
Keyed hash and MAC	BLAKE3 keyed mode	32 B key and tag
Key derivation	BLAKE3 keyed mode with XOF output	any
Signature	Ed25519, strict rules of 3.7.4	32 B key, 64 B sig
Key agreement, classical	X25519	32 B
Key agreement, post-quantum	ML-KEM-768	1184 B key, 1088 B ct
AEAD, counter nonce	ChaCha20-Poly1305	32 B key, 12 B nonce
AEAD, random nonce	XChaCha20-Poly1305	32 B key, 24 B nonce
Password hash	Argon2id	Appendix B
Handshake framework	Noise_IK_25519_ ChaChaPoly_BLAKE2b	
Transport, binding A	TLS 1.3 in QUIC, raw public keys	
Proof of work	Equi-X	16 B solution
Group key schedule	TreeKEM with the suite of 3.9.6	

Table 28. Mandatory primitives.

3.7.3 BLAKE3

BLAKE3 is the most consequential choice here, because the protocol uses it in four roles.

- **The tree structure gives verified streaming.** A recipient checks a 64 KiB chunk of a 100 MiB attachment against the root hash with a log-size proof, and rejects a bad chunk immediately. Attachments (3.9.9) depend on this. SHA-256 cannot do it, and a Merkle tree over SHA-256 would need a separate specification.
- **One function gives three modes.** Plain, keyed, and derive-key. SSP uses the keyed mode as both MAC and KDF, which removes HMAC and HKDF from the implementation, along with two more places to pick the wrong construction.

- **Speed on a weak device.** Several times faster than SHA-256 in software with no SHA extension. Battery cost matters when a client checks logs, hashes attachments, and derives mailbox addresses every day.
- **Extendable output.** SSP needs 32 bytes in one place and 64 in another; the XOF removes the counter loop HKDF-Expand needs.

The costs: BLAKE3 is younger than SHA-256 and less deployed, though its core is BLAKE2s, which has a long analysis record, and the tree mode has a published proof. It carries no NIST approval, so SSP cannot be used in an environment that requires one. Compliance is not a goal.

SSP-CRY-002 A client **MUST** use BLAKE3-256 for every hash in this specification, except inside the Noise handshake and inside TLS.

Rationale. One hash function for the protocol. 3.7.8 explains the exception.

Verification. Test.

Candidate	Reason for rejection
SHA-256	No tree mode, no keyed mode, slower in software, and needs HMAC and HKDF as separate constructions. Its only advantage is compliance status.
SHA-512, SHA-512/256	Faster on 64-bit hardware and slower on 32-bit hardware. The same missing modes.
SHA-3, SHAKE	A good XOF and much slower in software. The sponge offers no tree mode for verified streaming.
BLAKE2b	The direct ancestor. No tree mode in the common libraries and no XOF in the standard form.

Table 29. Hash functions considered.

3.7.4 Ed25519

Small keys and signatures, deterministic nonce generation, fast verification, no dependence on a random source at signing time, and a large set of audited implementations. Determinism removes the failure class that broke ECDSA in several deployed systems.

Known problem	Answer
Malleability, where a verifier accepts a non-canonical S	The strict rules below.
Libraries accept different signature sets, breaking agreement	One fully specified rule set.
No strong unforgeability under key reuse across contexts	A domain label in every signed object (SSP-GEN-007).
Not post-quantum	3.7.12 states the plan and the reason for the delay.

Table 30. Ed25519 problems and answers.

ECDSA over P-256 is rejected for its random nonce requirement and its point validation history; RSA for key size and parameter choice; Ed448 because it is slower and 128-bit classical security is sufficient under a plan that already covers the quantum threat.

SSP-CRY-007 An Ed25519 verifier MUST apply all of the following:

- (a) reject a signature whose scalar S is not fully reduced modulo the group order;
- (b) accept a public key A and a signature point R in a small-order subgroup, using the cofactored equation $[8][S]B = [8]R + [8][k]A$;
- (c) reject a public key or signature point that is not a valid, canonically encoded curve point;
- (d) reject a signature over an object whose encoding is not canonical.

Rationale. Rules (a) to (c) give one deterministic answer across every library implementing the ZIP 215 rule set, and remove the malleability that lets an attacker produce a second valid signature over one object. Rule (d) ties the signature rules to 3.5.4.

Verification. Test with the full ZIP 215 vector set and the vectors of Appendix E.

SSP-CRY-008 An implementation MUST NOT use batch verification for an identity log entry unless that mode yields the same accept set as rules (a) to (c) of SSP-CRY-007 for every input.

Rationale. Batch verification with a different accept set turns a log into a fork generator across implementations.

Verification. Test.

3.7.5 X25519

A fixed 32-byte encoding, no point format confusion, a safe curve whose cofactor the X25519 function handles, and universal library support.

SSP-CRY-003 A client MUST check that an X25519 output is not the all-zero value and MUST abort the operation when it is.

Rationale. A peer sending a small-order point would otherwise force a predictable shared secret. The check is one comparison, and its absence has broken deployed systems.

Verification. Test with the small-order point vectors of Appendix E.

3.7.6 ChaCha20-Poly1305

A software implementation is naturally constant-time, where AES needs table-free code to reach the same property and most libraries do not provide it. Performance on a device without AES-NI or the ARMv8 crypto extension is far better. The construction is simple, and nonce reuse costs the confidentiality of two messages and the authentication key, which is no worse than GCM, where reuse also gives forgery of every later message. The 256-bit key leaves a comfortable margin against a Grover-style attack.

SSP-CRY-004 A client MUST derive a fresh key for each AEAD invocation in the message path, and MUST use the nonce that 3.9.2 specifies.

Rationale. A fresh key per message makes the nonce question trivial and removes the counter management defects that cause reuse.

Verification. Test.

SSP-CRY-005 Where a nonce cannot come from a counter, a client **MUST** use XChaCha20-Poly1305 with a 24-byte random nonce.

Rationale. A 24-byte random nonce keeps the collision probability negligible for any realistic message count. A 12-byte random nonce does not.

Verification. Inspection of each call site.

AES-256-GCM is not in the mandatory set. A client **MAY** use it for the local store on a platform with hardware acceleration, where the key never crosses the network and the platform controls the nonce, and **MUST NOT** use it on the wire.

3.7.7 Argon2id

Memory hardness, resistance to both side-channel and time-memory tradeoff attacks, an RFC, and wide library support. Parameters are in Appendix B and target about one second on a mid-range 2024 phone. PBKDF2 is rejected as cheap on a GPU, scrypt because Argon2id supersedes it with clearer parameter guidance, and bcrypt for its password length limit and fixed small memory use.

3.7.8 Noise with BLAKE2b

The Noise framework names four hash functions: SHA-256, SHA-512, BLAKE2s, and BLAKE2b. BLAKE3 is not among them. SSP uses `Noise_IK_25519_ChaChaPoly_BLAKE2b` and defines no BLAKE3 variant. A new hash inside Noise would produce a pattern name no other implementation has, would discard the published Noise test vectors, and would move the handshake outside the analysis the Noise community has already done. The gain would be one fewer hash function in the binary; the loss would be the review lineage. BLAKE2b ships in every library that implements Noise, so the second hash costs no new dependency in practice.

3.7.9 QUIC with raw public keys

Binding A uses TLS 1.3 inside QUIC with the raw public key certificate type of RFC 7250. The relay authenticates with its Ed25519 relay key. There is no certificate chain, no certificate authority, and no name validation. A certificate authority is a trusted third party, and SSP has none. A relay identifier is a key, exactly as an identity is a key structure. The client learns the relay key from the relay set, which arrives inside an authenticated identity log or a contact code.

SSP-NET-003 A client **MUST** authenticate a relay by its Ed25519 relay key, and **MUST NOT** accept a relay on the basis of a certificate chain, a hostname, or a DNS record.

Rationale. A hostname is a name a registrar controls and a certificate chain is a trust set a browser vendor controls. Neither belongs in the trust model.

Verification. Test with a valid certificate for the correct hostname and the wrong key, which the client rejects.

3.7.10 Architectural alternatives rejected

Table 31. Cryptographic architectures considered and rejected.

Alternative	Reason
HPKE (RFC 9180) for messages	A clean single-shot public key encryption interface with no ratchet. SSP needs forward secrecy per message, so the ratchet is mandatory and HPKE would be an extra layer with no gain.
MLS as the whole protocol	Excellent, and it assumes a delivery service that totally orders handshake messages. That service is a central point. 3.9.7 keeps TreeKEM and removes the ordering service.
A triple ratchet or sender-key hybrid for two parties	The double ratchet suffices for two parties and has the longer analysis record.
Sender keys for groups	O(1) send cost, at the price of forward secrecy inside an epoch and a full redistribution on every membership change. TreeKEM gives logarithmic cost on membership change, which matters at 1024 members.
Onion routing inside SSP	Tor exists, has a decade of analysis, and has a network. 3.8.3 lets SSP run over it. A second onion network with a small user base would give worse anonymity than Tor with a large one.
Ring signatures or stealth addresses	These hide a sender in an anonymity set on a public ledger. SSP has no ledger, and its addressing layer already gives an opaque per-pair address, which is stronger here and far cheaper.
Post-quantum signatures now	3.7.12.

3.7.11 Key derivation

Every derivation has the same shape:

$$k = \text{BLAKE3}_{\text{keyed}}(k_{\text{parent}}, \text{SSP1} / \parallel \text{purpose} \parallel 0 \times 00 \parallel \text{context})$$

where the purpose comes from the registry in Appendix B and the context holds the values that make the derivation unique, canonically encoded.

Purpose	Context	Output
mailbox-root	sender SID, sender device, recipient SID, recipient device	32 B
mailbox-addr	epoch index	32 B
deposit-seed	epoch index	32 B
ratchet-root	DH output	64 B
msg-key	chain index	32 B
blob-key	blob root hash	32 B
store-key	device id	32 B

Table 32. Derivation examples.

SSP-CRY-006 An implementation MUST NOT use a label outside the registry of Appendix B, and MUST NOT reuse a label for a second purpose.
Verification. Inspection and Test.

3.7.12 Post-quantum plan

Confidentiality and authenticity have different deadlines. Confidentiality has a deadline in the past: an adversary records traffic today and decrypts it when a quantum computer exists. Authenticity has a deadline in the future: a signature broken in 2040 is worthless, because the message is old and the key long rotated. An attacker must break a signature before the moment of use.

SSP-CRY-009 Every session key agreement **MUST** combine X25519 and ML-KEM-768, and **MUST** derive the session secret from both outputs.

Rationale. The hybrid keeps classical security if ML-KEM-768 falls to cryptanalysis, and quantum security if X25519 falls to a quantum computer. Lattice schemes are young, and a hybrid is the only responsible choice at a ten-year horizon.

Verification. Test, plus analysis of the key schedule.

The combination is a hash of both secrets with the transcript:

$$ss = \text{KDF}(\text{SSP1/hybrid-combine}, ss_{\text{X25519}} \parallel ss_{\text{ML-KEM}} \parallel H_{\text{transcript}}, 64)$$

SSP-CRY-010 The hybrid combination **MUST** hash both secrets together with the handshake transcript, and **MUST NOT** use exclusive-or.

Rationale. A hash of both inputs is secure when either input is secure, and it binds the result to the transcript. Exclusive-or reaches the same security only under stronger assumptions about the independence of the two outputs.

Verification. Analysis.

Signature migration.

SSP-CRY-011 Every key record in an identity log **MUST** carry a two-byte algorithm identifier, and a verifier **MUST** reject an identifier it does not know.

Rationale. This one field is the entire migration mechanism. A future SSP/2 adds the ML-DSA-65 identifier, and an SSP/1 client meeting it fails closed rather than accepting an unverified key.

Verification. Test.

The procedure, when the maintainers trigger it: SSP/2 defines the ML-DSA-65 identifier and a dual-signature entry type; a client writes a RootRotate carrying both an Ed25519 key and an ML-DSA-65 key, signed by both; contacts that support SSP/2 use the newer key and older contacts use Ed25519; after the deprecation date a verifier rejects an Ed25519-only entry.

ML-DSA-65 costs 1952 bytes per public key and 3309 bytes per signature today. In a log with fifty entries and eight devices that is roughly 200 KiB of growth, which 3.12 does not accept while the classical scheme is sound.

3.7.13 Randomness

SSP-CRY-012 A client **MUST** take all random bytes from the operating system generator.

Rationale. A user-space generator adds a failure mode with no benefit; the kernel holds the entropy sources and the reseed logic.

Verification. Inspection.

SSP-CRY-013 A client **MUST NOT** generate a long-term key before the operating system reports that the generator is ready.

Rationale. An embedded device or a fresh virtual machine can boot with a predictable pool, and keys made in that window are guessable.

Verification. Test with a delayed entropy source.

SSP-CRY-014 A client **MUST NOT** use a deterministic generator seeded only by a user passphrase, except for the recovery key derivation of [3.6.10](#).

Verification. Inspection.

3.8 Networking, transport, and relays

3.8.1 Network model

The network holds clients and relays and nothing else: no supernode, no bootstrap authority, no distributed hash table, no overlay between relays.

A device holds a relay set of one to four relays. It deposits into the relay set of the peer and fetches from its own. Two devices need no common relay, because the sender connects to the relay of the recipient.

SSP-NET-004 A device **MUST** maintain a relay set of at least two relays run by different operators.

Rationale. One relay is a single point of failure and of observation. Two relays under one operator give neither redundancy nor privacy.

Verification. Inspection of client policy, and Demonstration.

SSP-NET-005 A sender **MUST** deposit a copy of the envelope at every relay in the relay set of the recipient device, unless it received an acknowledgement from one relay and the recipient acknowledged the message end-to-end within 30 seconds.

Rationale. A relay that drops data is inside T4. Duplicate deposits cost bandwidth and buy delivery under an unreliable or hostile relay. The recipient discards the duplicate by message identifier.

Verification. Test with a relay that drops silently.

3.8.2 Binding A: QUIC

Binding A is the default, and both clients and relays support it.

QUIC brings stream multiplexing without head-of-line blocking, so a client can fetch from two hundred mailboxes and upload a blob on one connection. Connection migration lets a phone move between mobile and wireless networks without a fresh handshake, which matters both for battery cost and for the metadata a new handshake reveals. Resumption puts a fetch in the first flight when a client wakes. Congestion control and loss recovery come free. The datagram extension gives a cheap channel for padding.

SSP-NET-006 A client and a relay **MUST** offer exactly one cipher suite and one key exchange group in the TLS handshake of binding A.

Rationale. It removes the downgrade surface and makes the handshake fingerprint identical across every SSP client, which supports [3.11.4](#).

Property	Value
Transport	QUIC version 1
Security	TLS 1.3 inside QUIC
Certificate type	Raw public key, Ed25519
ALPN	ssp/1
Cipher suite	TLS_CHACHA20_POLY1305_SHA256 only
Key exchange group	x25519 only
Client authentication	None at the TLS layer
Datagrams	RFC 9221, for padding and keepalive
Default port	4404/UDP

Table 33. Transport binding A.

Verification. Test with a packet capture.

SSP-NET-007 A client MUST NOT send an application payload in a 0-RTT flight that changes state at the relay.

Rationale. 0-RTT data is replayable by definition. A fetch is idempotent and safe; a deposit is not, and the correct rule is to keep replayable data out of that flight.

Verification. Test.

Candidate	Reason for rejection
TCP with custom framing and Noise, as the only binding	Multiplexing, flow control, and migration would all have to be written. Binding B does use this stack, and only where QUIC cannot go.
HTTPS and WebSocket	Adds an HTTP stack, an HTTP parser, and a header set that leaks a client fingerprint. It buys traversal of a strict proxy, which binding B with a pluggable transport also achieves.
Plain UDP with custom reliability	Rewriting QUIC, worse.
Tor as the only transport	Multi-second latency, bandwidth limits, and dependence on one network. 3.8.3 makes it an option instead.

Table 34. Transports considered.

3.8.3 Binding B: TCP with Noise IK

Binding B serves three cases: an onion service address, a network that blocks UDP, and a censored network that needs obfuscation.

SSP-NET-008 A relay MUST support binding B, and MUST publish both an Ed25519 relay key and an X25519 link key in its relay record.

Rationale. A relay reachable only over QUIC is unreachable from a network that blocks UDP and from an onion service, and the user in that position is the user who needs SSP most.

Verification. Demonstration.

Property	Value
Transport	TCP, or a pluggable transport presenting a stream
Handshake	Noise_IK_25519_ChaChaPoly_BLAKE2b
Initiator static	An ephemeral key per connection, never the device key
Responder static	The X25519 link key of the relay, from the relay set
Framing	A <code>u16</code> length prefix, then a Noise transport message
Maximum message	65 535 bytes, segmented by the frame layer
Default port	4404/TCP

Table 35. Transport binding B.

SSP-NET-009 In binding B a client **MUST** use a fresh ephemeral static key per connection, and **MUST NOT** present its device key or link key to the relay.

Rationale. The IK pattern authenticates the initiator to the responder, which SSP does not want. A stable client key would link every connection of that device across time and across networks. The client stays anonymous to the relay, and the mailbox deposit key supplies the authorisation instead.

Verification. Test with two connections from one client, which present different static keys.

SSP-NET-010 A client running over an onion network **MUST NOT** use binding A.

Rationale. QUIC needs UDP and an onion network gives a TCP stream. A client that tries binding A there either fails or falls back in a way an observer can see.

Verification. Test.

SSP-NET-011 A client **MUST** support a pluggable transport interface taking a stream and returning a stream, and **MUST NOT** define its own obfuscation protocol.

Rationale. Obfuscation is a fast-cycle arms race that the Tor project already runs with obfs4, Snowflake, and WebTunnel. A protocol that hard-codes one method carries a defeated method for years.

Verification. Inspection of the interface, and Demonstration with obfs4.

3.8.4 Relay link

A link opens with a transport handshake, then HELLO and HELLO_ACK. The acknowledgement carries the policy the relay enforces, and the client adapts to it.

SSP-RLY-001 A relay **MUST** state its full policy in HELLO_ACK, and **MUST NOT** enforce a limit it has not stated.

Rationale. A hidden limit produces a failure the client cannot diagnose and makes the network behave differently at each relay. G-01 needs the policy on the wire.

Verification. Test with a client driving every stated limit to its bound.

SSP-RLY-002 A relay **MUST NOT** request an identifier, an account, an email address, a payment token, or any value that persists across links.

Rationale. Every such value is a stable identifier linking the sessions of one device.

Verification. Inspection of the operation registry.

Field	Type	Meaning
max_mailboxes	u32	Mailboxes one link may register.
mailbox_ttl	u32	Seconds until an unrenewed mailbox expires.
object_ttl	u32	Seconds until a stored object expires.
max_object_bytes	u32	Largest envelope.
default_quota	u32	Objects per mailbox per 24 hours.
max_quota	u32	Largest quota a mailbox may request.
blob_max_bytes	u64	Largest blob.
blob_ttl	u32	Seconds until a blob expires.
pow_algorithm	u16	0x0001 for Equi-X.
pow_effort_intro	u32	Current effort for an introduction deposit.

Table 36. Relay policy block.

Mailbox operations. Registration carries the address, the deposit public key, the quota in objects and bytes, a class, a lifetime, and a signature by the deposit secret proving that the registrant holds the key being registered.

SSP-RLY-003 A relay MUST reject a deposit to an address that no registration created.

Rationale. This single rule ends open relaying. A relay stores data only for a mailbox a subscriber asked for, so an attacker cannot make a relay store data by inventing addresses, and the relay cannot become a free disk for the internet.

Verification. Test.

SSP-RLY-004 A relay MUST require a valid signature by the deposit key on registration, renewal, and deletion.

Rationale. Otherwise any party learning a mailbox address could delete the mailbox of the victim, which is a cheap denial-of-service attack.

Verification. Test.

The asymmetry is the point. The recipient registers the mailbox and proves it holds the deposit secret, then hands that secret to exactly one sender and keeps only the public key. From then on the sender can deposit, the recipient has a per-sender revocation switch through deletion, and the relay has a cheap check.

Fetch and subscribe. A subscription or a fetch names a set of mailboxes and a cursor per mailbox; the relay returns envelopes with a sequence number and a storage time, and deletes on acknowledgement.

SSP-RLY-005 A relay MUST assign a strictly increasing sequence number per mailbox and MUST NOT reuse a value.

Rationale. The cursor model needs it, and a gap tells a client that the relay dropped or deleted an object.

Verification. Test.

SSP-RLY-006 A relay MUST delete an object on a valid acknowledgement and MUST NOT retain a copy.

Rationale. The value of deletion is that a later seizure of the disk finds nothing. A relay that ignores this is inside T4, and no confidentiality property depends on the rule.

Verification. Demonstration on an honest relay. The protocol does not depend on it.

SSP-RLY-007 A client **MUST** acknowledge only after writing the plaintext to durable local storage.

Rationale. An acknowledgement before a durable write loses the message on a crash.

Verification. Test with crash injection.

3.8.5 What a relay knows

Table 37 is normative. An implementation that lets a relay learn more than the left column is non-conformant.

A relay knows	A relay never knows
The IP address of the connecting client, absent an onion network or VPN	Which identity that client belongs to
The set of mailbox addresses one link registers or reads	Which peer, group, or conversation an address serves
The size class of each object, from the bucket field	The plaintext length inside the bucket
The type class of each object	The content type of the message
Deposit time, fetch time, object count	The identity of the depositor
The deposit key of a mailbox	The identity holding the matching secret
Blob chunk hashes and sizes	Blob plaintext, and the message referencing a blob
The identity logs a client asks it to serve	Nothing further; a log is public to any contact

Table 37. The relay knowledge boundary.

SSP-RLY-008 A relay **MUST NOT** record a mapping between an IP address and a mailbox address in persistent storage.

Rationale. An operator can break this rule, and the rule still has value: it defines conformance and gives an honest operator a clear instruction.

Verification. Inspection of the reference relay, and Demonstration.

SSP-RLY-009 A relay **MUST NOT** place a mailbox address in a log file, an access log, or a metrics label.

Rationale. Metrics systems retain data for months and often ship it off the host.

Verification. Inspection.

3.8.6 Mailboxes

The core privacy mechanism is this: the network address of a recipient is not a property of the recipient, but of the pair.

Signal sends to a phone number, Matrix to a user on a homeserver, Session to a public key. In each case one identifier receives all traffic for one user and every sender uses it, so the operator can count the senders of a user, measure their traffic, and hold a subpoena-ready record. SSP derives a distinct address for each sender-device and recipient pair and rotates it daily. No two senders of the same recipient use the same address, and no address survives a day.

Derivation. At session setup both parties compute

$$mailbox_root = \text{KDF}(ss, \text{SSP1/mailbox-root}, \text{SID}_s \parallel dev_s \parallel \text{SID}_r \parallel dev_r, 32)$$

and thereafter, with no exchange of messages,

$$\begin{aligned} e &= \lfloor t_{ms}/86\,400\,000 \rfloor \\ addr_e &= \text{KDF}(mailbox_root, \text{SSP1/mailbox-addr}, e, 32) \\ seed_e &= \text{KDF}(mailbox_root, \text{SSP1/deposit-seed}, e, 32) \\ (dsk_e, dpk_e) &= \text{Ed25519.KeyGen}(seed_e) \end{aligned}$$

The recipient registers the next address one epoch ahead; the sender computes the same values and deposits.

SSP-MDR-001 A mailbox address **MUST** change at every epoch boundary, and the new address **MUST NOT** be computable from the old one without the mailbox root.

Rationale. An observer watching a relay for a year must not be able to link day 1 to day 365 for the same pair.

Verification. Test with the vectors of Appendix E.

SSP-MDR-002 A recipient **MUST** register the mailbox of epoch $e+1$ before the end of epoch e , and **MUST** keep the mailbox of epoch $e-1$ for a further 24 hours.

Rationale. Clock skew of a few minutes must not lose a message; a 24-hour overlap covers assumption A-02 with a wide margin.

Verification. Test with a client whose clock is ten minutes fast.

SSP-MDR-003 A recipient **MUST** register the mailboxes of an epoch in random order, spread across the epoch.

Rationale. A relay receiving two hundred registrations in one burst learns that one client holds two hundred mailboxes. Spreading hides the count at the cost of more link time.

Verification. Demonstration, and analysis of the timing distribution.

SSP-MDR-004 A recipient device **MUST** use one mailbox per peer *identity* for the receive direction, not one per peer device.

Rationale. The devices of one peer already share that identity and already know the SID, so a per-device receive mailbox adds no unlinkability against a relay and multiplies the count by eight. The send direction still uses a per-device session, because the ratchet is per device.

Verification. Inspection and Analysis.

With that rule a user with two hundred contacts holds two hundred receive mailboxes per device, plus one per group and one introduction mailbox.

3.8.7 Relay storage and expiry

SSP-RLY-010 A relay **MUST** delete an object when its expiry passes, and **MUST NOT** keep a copy or a hash of it.

Verification. Demonstration.

Class	Max object	Default expiry	Max expiry
Message	64 KiB	14 days	30 days
Introduction	8 KiB	7 days	7 days
Group commit	256 KiB	30 days	30 days
Blob notice	4 KiB	14 days	30 days
Receipt	1 KiB	3 days	7 days
Blob chunk group	1 MiB	7 days	14 days

Table 38. Storage classes at a relay.

SSP-RLY-011 A relay **MUST** hold a group commit object for at least 30 days.

Rationale. A member offline for three weeks must still be able to reach the current epoch. Shorter retention forces a full rejoin, which costs the group a commit and leaks the absence of that member.
Verification. Test.

SSP-RLY-012 A relay **MUST** enforce a total byte limit per mailbox and reject a deposit beyond it with the reason `MAILBOX_FULL`.

Rationale. A per-object limit alone lets a sender fill a disk with many objects.
Verification. Test.

SSP-RLY-013 A relay **MUST NOT** delete an unacknowledged object before its expiry to make space, and **MUST** reject the new deposit instead.

Rationale. Dropping the oldest object gives an attacker a way to push a message out of the mailbox of a victim: a targeted deletion attack.
Verification. Test.

3.8.8 Relay discovery

A relay record holds the Ed25519 relay key, the X25519 link key, a list of endpoints (IPv4, IPv6, DNS name, or onion address), flags for whether it accepts new mailboxes and blobs, and a signature over the record and a validity window.

A client learns of a relay in four ways and no others: a bootstrap list embedded at build time; a record the user pastes or scans; up to four records inside a contact code; or a signed relay directory the user chooses to follow. A directory is a signed, hash-linked document that anyone may publish. It is not a protocol service, and the signature of the publisher is its only authority.

SSP-NET-012 A client **MUST** let the user replace the entire bootstrap relay list.

Rationale. A fixed bootstrap list is a central point a state adversary can block and a client author can abuse.
Verification. Demonstration.

SSP-NET-013 A client **MUST NOT** trust a relay directory for anything beyond the existence and the address of a relay.

Rationale. A directory publisher must not be able to affect confidentiality. It can affect availability, and 3.8.9 states the consequence.

Verification. Analysis.

No distributed hash table. A DHT would give automatic discovery and user lookup, and is rejected for both.

Problem	Detail
Sybil attacks	With no admission control, an attacker places nodes near a target key and owns the lookups for it. Every known defence needs a trusted party, proof of work, or a stake.
Enumeration	A DHT storing user records is a queryable list of every user.
Metadata	A lookup tells the nodes on the path which key the client wants: the social graph, published to strangers.
Complexity	A production DHT runs to thousands of lines with a long defect history in routing, traversal, and churn.

Table 39. Reasons against a DHT.

SSP therefore has no lookup by identifier. A contact code carries the relay hints and the identity log carries updates. If every relay of a contact disappears at once, that contact must send a new code out of band. This is a real loss of convenience and an accepted one.

SSP-MDR-005 The protocol **MUST NOT** define any operation mapping a human-visible value, a SID, or a hash of either, to a network address, a relay, or a mailbox.

Rationale. Every such operation is a user enumeration oracle. Rate limits do not fix it, because an attacker with a botnet has unlimited query budget. Signal built a hardware enclave for this problem and the result still depends on the enclave vendor. SSP removes the problem instead of managing it.

Verification. Inspection of the operation registry in Appendix C.

3.8.9 Relay properties

SSP-RLY-014 A relay **MUST** hold no state that another relay cannot recreate from client actions.

Rationale. A user must be able to leave a relay in one minute, losing only the objects in flight. This is what keeps an operator honest and stops a relay becoming a platform.

Verification. Demonstration of a relay migration.

The useful question is not whether a relay is trustworthy but what it gains by being hostile. Table 40 answers it.

No federation. A relay never talks to another relay.

The cost is that a sender must reach the relay of the recipient. Where a censor blocks that relay, the sender uses binding B with a pluggable transport.

3.9 Messaging

Compromise	Effect
The operator reads the disk	Ciphertext, opaque addresses, deposit times. No content and no identities.
The operator seizes a running process	The same, plus live IP addresses and the mailbox sets of current links.
The relay drops every object	Denial of service for its users. Two relays are required, so one relay dropping does not stop delivery.
The relay serves a forked identity log	Detected: clients cross-check two relays and gossip heads.
The relay lies about its policy	The client sees failures and moves.
The relay colludes with a network observer	The T5 and T6 combination; 3.11.6 states the limit.

Table 40. Effect of relay compromise.

Reason	Detail
No routing state	Federation needs a routing table, and a routing table is a map of where users are.
No amplification	A relay forwarding to another relay is a reflector and a flood vector.
No trust edges	Each relay would have to decide which relay to trust, recreating the reputation problem that 3.10 rejects.
Simplicity	The sender already knows the relay set of the recipient; a forward adds a hop for nothing.
Evidence	Matrix federation replicates room state to every participating server, multiplying metadata exposure by the server count.

Table 41. Reasons against federation.

3.9.1 Asynchronous key agreement

SunSpot asynchronous key agreement (SAKA) establishes a session between two devices while the recipient is offline. It follows the shape of PQXDH and uses the SSP primitive set.

Prekey bundle. A device publishes a bundle to a prekey mailbox at each of its relays, named by a PrekeyPointer entry in its identity log.

SSP-MSG-004 A relay **MUST** give each one-time prekey to at most one requester, and **MUST** delete it once given.

Rationale. The one-time key is what makes the first message forward-secret. A relay serving the same key twice destroys that property for both senders.

Verification. Test with two concurrent requesters.

SSP-MSG-005 A sender **MUST** verify every signature in a bundle against the device key held in the identity log of the recipient, before using any value from it.

Rationale. The bundle arrives from an untrusted relay. Without the check, the relay chooses the keys and reads everything.

Verification. Test with a tampered bundle.

The agreement. The initiator holds an ephemeral EK_i and its static link key L_i ; the responder has published SPK_r , an optional one-time OPK_r , its static link key L_r , a static KEM key, and an optional one-time KEM key.

Field	Type	Lifetime
device id	[8]	long term
identity key	Ed25519 [32]	long term
link key	X25519 [32]	180 days
KEM key	ML-KEM-768 [1184]	180 days
signed prekey	X25519 [32]	7 days
one-time prekey	X25519 [32]	single use
one-time KEM key	ML-KEM-768 [1184]	single use
signatures	Ed25519 [64]	over each element and over the whole bundle

Table 42. Prekey bundle.

$$\begin{aligned}
dh_1 &= \text{X25519}(EK_i, SPK_r) & dh_2 &= \text{X25519}(L_i, SPK_r) \\
dh_3 &= \text{X25519}(EK_i, L_r) & dh_4 &= \text{X25519}(EK_i, OPK_r) \text{ if present} \\
(c_1, k_1) &= \text{MLKEM768.Encaps}(KEM_r) & (c_2, k_2) &= \text{MLKEM768.Encaps}(OKEM_r) \text{ if present}
\end{aligned}$$

$$\begin{aligned}
T &= H(\text{SSP1/saka-transcript} \parallel \text{SID}_i \parallel dev_i \parallel \text{SID}_r \parallel dev_r \parallel \\
&\quad EK_i \parallel SPK_r \parallel OPK_r \parallel KEM_r \parallel OKEM_r \parallel c_1 \parallel c_2) \\
ss &= \text{KDF}(dh_1 \parallel dh_2 \parallel dh_3 \parallel dh_4 \parallel k_1 \parallel k_2, \text{SSP1/saka-root}, T, 64)
\end{aligned}$$

The initial message carries the ephemeral public key, the KEM ciphertexts, the identifiers of the prekeys used, and the first ratchet message. It goes to the introduction mailbox of the recipient, or to an existing mailbox if the pair has met before.

SSP-MSG-006 SAKA MUST include dh_2 , which uses the static link key of the initiator.

Rationale. dh_2 authenticates the initiator cryptographically. Without it, any party could open a session as any sender, and the recipient would need a signature over the whole message, which would destroy deniability.

Verification. Analysis.

SSP-MSG-007 SAKA MUST NOT sign the initial message with the device key of the sender.

Rationale. Deniability. Authentication comes from dh_2 , which both parties can compute, so neither can prove authorship to a third party. A signature would be a transferable proof and would turn every message into evidence.

Verification. Inspection and Analysis.

SSP-MSG-008 The transcript hash MUST cover every public value of the handshake, and the session secret MUST bind to it.

Rationale. Downgrade protection and resistance to unknown key share.

Verification. Test: a modified transcript yields a different key.

X3DH alone has no post-quantum component. Noise has excellent interactive patterns, which SSP uses at the relay link and at enrolment, and it does not cover the asynchronous case where the responder is offline and a third party holds its prekeys. SAKA fills that gap and borrows the transcript discipline of Noise.

3.9.2 Double ratchet

Parameter	Value
DH function	X25519
KDF	BLAKE3 keyed mode
AEAD	ChaCha20-Poly1305
Nonce	The 12-byte big-endian chain index, with eight leading zero bytes
Root step	KDF(root, SSP1/ratchet-root, dh, 64) into a new root and chain
Chain step	KDF(chain, SSP1/ratchet-chain, i, 64) into the next chain and a message key
Header key	Derived per chain; headers are encrypted
Skipped keys	1000 per chain, 2000 per session
Chain length before a forced DH step	100

Table 43. Ratchet parameters.

SSP-MSG-009 A client MUST use the header encryption variant of the double ratchet.

Rationale. A cleartext header carries the ratchet public key and the counters. That key changes per step and would let a party reading two envelopes link them to one session. The header sits inside the mailbox ciphertext, so this defends against a compromised recipient store rather than the relay, at negligible cost.

Verification. Test.

SSP-MSG-010 A client MUST store at most 1000 skipped message keys per chain and drop the oldest beyond that.

Rationale. An attacker sending a header with a counter of 2^{31} would make a naive client derive two billion keys. The limit turns that attack into a lost message.

Verification. Test with a large counter jump.

SSP-MSG-011 A client MUST perform a DH ratchet step on the first message it sends after receiving one, and after 100 messages in one chain even with no reply.

Rationale. The DH step is what delivers post-compromise security. A long one-sided chain leaves a compromise open for its whole length.

Verification. Test.

SSP-MSG-012 A client MUST delete a message key immediately after use, and a skipped key after seven days.

Rationale. Forward secrecy is a property of deletion, not of the algorithm.

Verification. Inspection, and Test against a store dump.

3.9.3 Message object

The header carries a domain label, the version, the sender SID and device, a conversation identifier, the message identifier, a per-sender counter, a timestamp, one to eight parent identifiers, the current identity log head of the sender, and flags. The body carries a content type and a payload, followed by an extension list.

SSP-MSG-013 A message identifier **MUST** be the BLAKE3 hash of the canonical encoding of the message with the identifier field set to zero.

Rationale. Content addressing. The identifier becomes verifiable and no party can choose it; a random identifier would let a sender collide two messages deliberately.

Verification. Test.

SSP-MSG-014 A recipient **MUST** reject a message whose sender SID and device do not match the session that decrypted it.

Rationale. The ratchet supplies authentication; this check stops a confused sender identity inside a valid session.

Verification. Test.

SSP-MSG-015 A recipient **MUST** treat the timestamp as advisory, **MUST NOT** order by it, and **MUST** reject a message timed more than 24 hours in the future.

Rationale. The clock of the sender is not trusted, and the rejection window stops a message that would sort to the top of a list for ever.

Verification. Test.

3.9.4 Ordering and convergence

SSP gives causal consistency with eventual convergence, and no total order. No server can supply one, and a consensus mechanism is a non-goal.

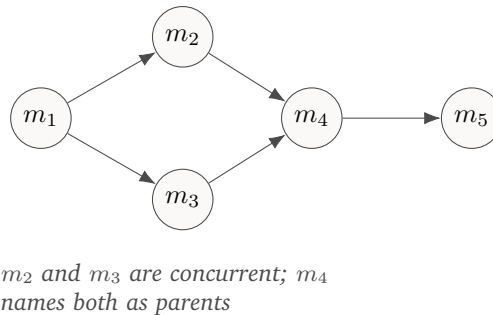


Figure 12. Messages form a directed acyclic graph. Each message names the heads its sender had seen.

SSP-MSG-016 A sender **MUST** name as parents every current head of its local view of the conversation, up to eight.

Rationale. The parent set proves what the sender had seen. It makes causality explicit and verifiable and lets a recipient detect a gap.

Verification. Test.

SSP-MSG-017 Where a conversation has more than eight heads, a sender **MUST** choose the eight with the highest identifiers in byte order.

Rationale. A deterministic rule means two clients in the same state produce the same parent set, which helps deduplication and testing.

Verification. Test.

SSP-MSG-018 A client **MUST** display messages in a topological order of the graph, breaking ties between concurrent messages first by the lower value of $\lfloor sent_at/60000 \rfloor$, then by the lower identifier in byte order.

Rationale. The minute bucket uses the sender clock where it is plausible, which matches user expectation, and the hash tie-break makes the result identical on every device, which is what convergence needs. A pure hash order would scramble a conversation; a pure timestamp order would let a sender with a wrong clock control the view.

Verification. Test with the convergence vectors of Appendix E.

SSP-MSG-019 A client **MUST NOT** hide a message because a parent is missing, and **MUST** show the gap.

Rationale. A relay inside T4 can drop a message. A client that waits for the parent for ever hands the attacker a silent censorship tool; a visible gap tells the user.

Verification. Demonstration.

Mutable state. Different state needs different rules, fixed here so that two clients converge.

State	Rule	Reason
Message body	Immutable	It is content-addressed.
Edit	Last writer wins, among edits by the original sender only	An edit by another party is a forgery.
Delete for me	Local only, never on the wire	A display choice.
Delete for everyone	A request, not a command; the client hides the content and keeps the tombstone	A recipient device can ignore it and the protocol must not pretend otherwise.
Reaction	Add-wins set keyed by sender and emoji	Concurrent add and remove converge to add, matching what a user sees.
Profile	Last writer wins per identity	One value, one owner.
Group name and topic	Last writer wins per epoch; a commit resets it	The epoch is a natural ordering point.
Read state	Grow-only counter per conversation per device	Read state only moves forward.

Table 44. Convergence rules for mutable state.

SSP-MSG-020 A delete-for-everyone request **MUST NOT** remove the tombstone, and a client **MUST** show that a message existed and was withdrawn.

Rationale. An honest client cannot prove it deleted the content, and a protocol promising deletion makes a promise TX-03 excludes. A visible tombstone gives the user the truth.

Verification. Demonstration.

3.9.5 Delivery and receipts

Deposited means a relay stored the envelope; delivered means a recipient device decrypted the message; read means the user saw it.

SSP-MSG-021 A delivery receipt **MUST** travel end-to-end as a message, and **MUST NOT** come from a relay.

Rationale. A relay receipt would be a lie, because a relay cannot know that a device decrypted anything, and it would give the relay a reason to hold a mapping.

Verification. Inspection.

SSP-MSG-022 A client **MUST NOT** send a read receipt unless the user enabled it for that conversation.

Rationale. A read receipt is precise activity metadata sent to a party who may not deserve it.

Verification. Demonstration.

SSP-MSG-023 Delivery **MUST** be at-least-once, and a recipient **MUST** deduplicate by message identifier.

Rationale. Deposits at two relays are normal, so duplicates are normal.

Verification. Test.

3.9.6 Groups

A group uses TreeKEM from RFC 9420 with the SSP primitive set, and without the MLS delivery service and authentication service.

MLS component	SSP replacement
Delivery service ordering handshake messages	None. 3.9.7 resolves concurrent commits deterministically.
Authentication service with credentials	The identity log. A leaf holds a SID and a device id, verified against that log.
Ciphersuite registry	One suite.
Public messages on a shared channel	Per-pair mailboxes, so a relay never sees a group.

Table 45. Divergence from MLS.

The suite is TreeKEM over X25519 with ML-KEM-768 combined as in 3.7.12, BLAKE3 as KDF and hash, ChaCha20-Poly1305 as AEAD, and Ed25519 for signatures.

SSP-GRP-001 A group **MUST** use exactly one ciphersuite, the one above.

Verification. Inspection.

SSP-GRP-002 A leaf of the tree **MUST** be a device, not an identity.

Rationale. A device holds the private key, so the tree must reach the device. A leaf per identity would require the devices of that identity to share a key, which SSP-DEV-003 forbids.

Verification. Inspection.

SSP-GRP-003 A group **MUST NOT** hold more than 1024 identities or 4096 devices.

Rationale. Commit size grows with the logarithm of the tree and mailbox fan-out grows linearly. At 4096 leaves a commit costs about twelve encrypted path secrets per leaf and one message costs 4096 deposits, which is the upper bound 3.12 accepts. Larger audiences use a channel.

Verification. Test.

SSP-GRP-004 A member **MUST** send an update proposal for its own leaf at least every seven days, and **MUST** commit at least every 30 days while the group is active.

Rationale. Post-compromise security in a group needs key refresh. Without a rule, a quiet group holds one secret for years and a single old compromise reads everything.

Verification. Test.

SSP-GRP-005 A commit that removes a member **MUST** replace every path secret from that leaf to the root.

Rationale. This is the TreeKEM property that gives forward silence after removal.

Verification. Test with a removed member attempting to decrypt the next epoch.

SSP-GRP-006 A member **MUST** check that the leaf credential of every other member matches a current device in the identity log of that member, and **MUST** reject a commit adding a leaf for a revoked device.

Rationale. The tree carries key material and the log carries the authority. A tree that disagrees with a log is a compromise.

Verification. Test.

Group traffic never reveals a group.

SSP-GRP-007 A group message **MUST** travel to each recipient device through the pairwise mailbox of 3.8.6, and **MUST NOT** use a shared group mailbox.

Rationale. A shared mailbox would show a relay the exact membership, size, and activity of the group, which is asset 3. The cost is fan-out: one message becomes N deposits.

Verification. Analysis and Test.

Fan-out is the price of the property. A group of fifty members with two devices each costs one hundred deposits of equal size; at the 4 KiB bucket that is 400 KiB of upload for one message. This is the first of the three open questions of 1.3, and 6 names the research that would improve it.

3.9.7 Concurrent commits

Two members may commit at the same epoch. MLS solves this with a delivery service that orders commits; SSP has none and needs a deterministic rule.

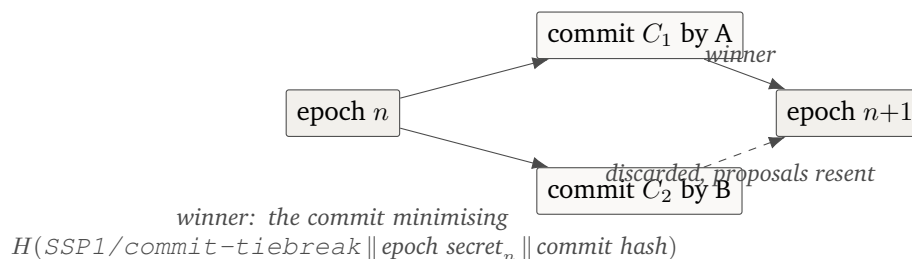


Figure 13. Deterministic resolution of concurrent commits.

SSP-GRP-008 A member **MUST** resolve concurrent commits at one epoch by the lowest value of $H(\text{SSP1/commit-tiebreak} \parallel \text{epoch_secret}_n \parallel \text{commit_hash})$.

Rationale. The rule is deterministic, so every member converges. Including the epoch secret means a committer cannot grind a hash to win, because the value depends on a secret that is common to all members and controlled by none.

Verification. Test with the concurrency vectors of Appendix E.

SSP-GRP-009 A member **MUST** retain epoch n state until it has seen no new commit for that epoch for 60 seconds, and **MUST** remain able to decrypt a message sent under epoch n for seven days.

Rationale. A member deleting the old key at once cannot read a message that crossed the epoch boundary. Seven days bounds the forward secrecy loss.

Verification. Test.

SSP-GRP-010 A member whose commit lost **MUST** resend its proposals in the next epoch, and **MUST NOT** resend a proposal the winning commit already applied.

Verification. Test.

The cost is one wasted round trip when two members commit at the same moment, which is rare outside a burst of membership change. The gain is that no party orders the group, and therefore no party can censor a commit by refusing to order it.

3.9.8 Channels

A channel is a one-to-many stream with one publisher and any number of subscribers. It exists because a group caps at 1024 identities and because a broadcast has different privacy needs.

Property	Group	Channel
Membership confidentiality	Hidden from relays and non-members	The publisher knows every subscriber
Who may send	Any member	The publisher only
Key structure	TreeKEM tree	A forward-ratcheted chain of epoch keys
Member limit	1024 identities	No protocol limit
Forward secrecy	Per message and per epoch	Per epoch, 24 hours by default
Removal	A commit rekeys the tree	The publisher stops issuing the next epoch key

Table 46. Groups compared with channels.

SSP-GRP-011 A channel epoch key **MUST** be derived as $k_{n+1} = \text{KDF}(k_n, \text{SSP1/channel-epoch}, \text{channel_id} \parallel n+1, 32)$, and the publisher **MUST** issue k_{n+1} only to the subscribers it keeps.

Rationale. A dropped subscriber holds k_n and the KDF is one-way, so it computes nothing forward. A joining subscriber receives only k_n and cannot read epoch $n - 1$.

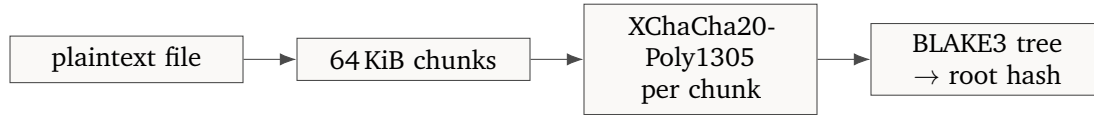
Verification. Test.

SSP-GRP-012 A client **MUST** show the user that a channel does not hide the subscriber list from the publisher.

Rationale. The property differs from a group and the user must know.

Verification. Demonstration.

3.9.9 Attachments



the message carries the root hash, the blob key, the true size, the chunk count, the content type, and relay hints

Figure 14. Attachment pipeline.

SSP-ATT-001 A blob MUST use a fresh random key.

Rationale. A shared key lets a party holding one key read others, and it breaks the deduplication privacy of SSP-ATT-005.

Verification. Test.

SSP-ATT-002 A recipient MUST verify each chunk against the root hash with the BLAKE3 tree proof, before decrypting it and before writing it to storage.

Rationale. This is the property BLAKE3 supplies and a flat hash cannot. A recipient downloading 100 MiB from a hostile relay detects a corrupt or malicious chunk at once rather than after the whole transfer.

Verification. Test with a corrupted chunk at the start, the middle, and the end.

SSP-ATT-003 A client MUST pad the final chunk to 64 KiB with random bytes and carry the true size inside the encrypted message.

Rationale. The final chunk size would otherwise leak the exact file size modulo 64 KiB, and an exact size is a strong fingerprint of a known file.

Verification. Test.

SSP-ATT-004 A blob MUST NOT exceed 100 MiB.

Rationale. A relay must bound its disk cost and a client its retry cost. A larger transfer belongs to a file transfer application, and a message protocol should not become one.

Verification. Test.

SSP-ATT-005 A client MUST NOT deduplicate a blob across recipients by reusing the root hash and key.

Rationale. Deduplication across recipients gives a relay a matching identifier in two mailboxes, linking two conversations. The bandwidth saving is not worth the link. Deduplication inside one conversation is permitted, where the relay already sees one mailbox.

Verification. Inspection and Test.

SSP-ATT-006 A client MUST strip image, audio, and video metadata before sending, unless the user disables it for that file.

Rationale. EXIF holds location, device model, and time. This is a protocol requirement rather than a product choice, because a client that omits it harms the recipient as well as the sender.

Verification. Test with a file carrying EXIF data.

3.9.10 Multi-device synchronisation

The devices of one identity form a self group: an ordinary group in which every device of the identity is a member and no other identity is. A DeviceAdd entry triggers an add proposal and a commit; a DeviceRemove triggers a remove proposal and an immediate commit.

SSP-SYN-001 A client **MUST** mirror every outgoing message, read state change, and contact change to the self group.

Rationale. Multi-device consistency with no server-side account. The self group offers the same forward secrecy and removal semantics as any group.

Verification. Test with three devices.

SSP-SYN-002 A client **MUST NOT** send plaintext state to a relay for synchronisation, and **MUST NOT** use a relay as a state store beyond the envelope model.

Rationale. A synchronisation store at a relay is an account, and an account is a central point holding long-lived data.

Verification. Inspection.

SSP-SYN-003 History transfer to a new device **MUST** be device-to-device, over the enrolment channel or a fresh self-group session.

Rationale. N-04. A server-side history store needs a long-lived key, which destroys forward secrecy for the whole archive. The transfer is a one-time cost at enrolment and can move gigabytes over a local network.

Verification. Demonstration.

SSP-SYN-004 A user **MUST** be able to choose how much history to transfer, with at least none, the last 30 days, and everything.

Rationale. A new device is a new copy of the archive, and the user must be able to limit the exposure.

Verification. Demonstration.

SSP-SYN-005 A client **MUST** queue an outgoing message when no relay of the recipient is reachable, retry with exponential backoff, and **MUST NOT** drop it before seven days pass.

Verification. Test.

SSP-SYN-006 A client **MUST NOT** change the parent set of a queued message when it retries.

Rationale. The parent set records what the sender had seen at the moment of writing; updating it on retry would rewrite causality.

Verification. Test.

3.9.11 Local storage

SSP-STO-001 A client **MUST** encrypt every persistent store with a key derived from the device master key.

Verification. Inspection.

SSP-STO-002 The device master key **MUST** be wrapped by the operating system key store, or by an Argon2id derivation from a user passphrase, and **MUST NOT** be stored in plaintext.

Verification. Inspection.

SSP-STO-003 A client **MUST** implement crypto-erase, where deleting a conversation deletes its keys, and **MUST NOT** depend on the file system to erase data.

Rationale. A modern file system and a solid-state disk do not overwrite in place. The only reliable deletion is deletion of the key.

Verification. Demonstration against a raw disk image.

SSP-STO-004 The local store **MUST** hold an append-only event log, and every view **MUST** be a materialised projection of it.

Rationale. The message graph is the truth. A store holding only the current view cannot rerun the ordering rule when a late message arrives, and cannot converge.

Verification. Inspection and Test with a late arrival.

SSP-STO-005 A client **MUST** support a retention policy per conversation, with at least keep everything, keep one year, keep 30 days, and keep nothing after reading.

Rationale. The largest realistic threat to most users is device seizure, and the best defence is to hold less data.

Verification. Demonstration.

3.10 Abuse prevention

SSP replaces reputation with capability. A relay accepts a deposit only into a mailbox a recipient created, and only under a signature by the deposit key that recipient chose. The recipient gives that key to one sender. Spam from an unknown party is therefore impossible by construction, except at the one place where a stranger must be able to reach a user.

3.10.1 Known senders

SSP-ABU-001 A relay **MUST** reject a deposit that does not carry a valid signature by the registered deposit key of the target mailbox.

Verification. Test.

SSP-ABU-002 A recipient **MUST** set a quota on each mailbox at registration, in objects per 24 hours and in total bytes, and a relay **MUST** enforce both.

Rationale. The recipient knows how much traffic a contact should send; the relay enforces the number without knowing who either party is.

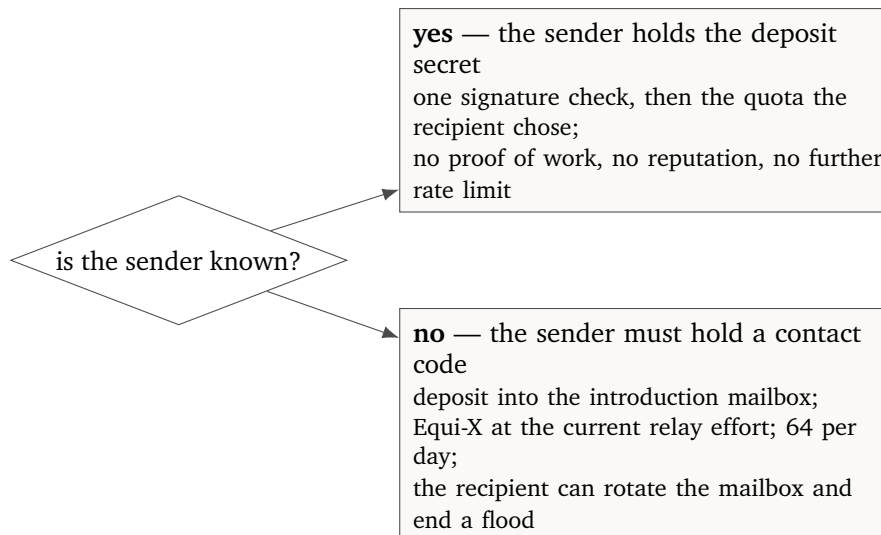


Figure 15. Admission decision at a relay.

Verification. Test.

SSP-ABU-003 A relay **MUST** answer a full quota with an explicit rejection carrying the reason and the time of the next reset, and **MUST NOT** drop silently.

Rationale. A silent drop looks like a network failure, so the sender retries for ever and wastes the resources of both parties.

Verification. Test.

SSP-ABU-004 A recipient that wants to stop a sender **MUST** delete that mailbox at every relay in its relay set.

Rationale. This is the block operation: instant, needing no server policy and no report to anyone. The blocked sender learns only that the mailbox is gone, which is indistinguishable from a relay change.

Verification. Test.

This yields a property no reputation system can offer: a block enforced by an untrusted relay, with no knowledge of who is blocked, no appeal process, and no operator judgement.

3.10.2 Unknown senders

An introduction mailbox is the only place a party with no prior relationship can deposit. It is long-lived, it appears in the contact code, and it is the whole spam surface of the protocol. Three mechanisms protect it.

The contact code is a secret. An attacker cannot enumerate introduction mailboxes, because [SSP-MDR-005](#) removes every lookup. The attacker must hold a code the user issued, which alone removes the mass-spam model of email and SMS.

Proof of work.

SSP-ABU-005 A relay **MUST** require a valid Equi-X solution for every introduction deposit, and **MUST** publish the current effort.

Verification. Test.

The challenge is $H(\text{SSP1}/\text{pow}||\text{relay_key}||\text{mailbox}||e||\text{relay_seed})$, and the solver finds a 16-byte solution that Equi-X accepts and whose hash falls below the target for the current effort.

Property	Detail
Asymmetric cost	A solution takes roughly five milliseconds of CPU and a few megabytes; verification takes about fifty microseconds. A relay under attack does not spend more than the attacker.
Memory hardness	Resists a hardware advantage, so a botnet of commodity CPUs has no worse ratio than a specialist attacker.
Deployment record	Tor runs it against funded denial-of-service attacks on onion services.
Implementation	A reviewed Rust implementation exists inside the Arti project.

Table 47. Why Equi-X.

A hashcash-style SHA-256 puzzle is rejected because specialist hardware gives an attacker a factor of 10^5 over a phone. Argon2id as a puzzle is rejected because verification costs as much as solution, which makes the relay the victim. A verifiable delay function is rejected because it needs a setup and gives no memory hardness.

SSP-ABU-006 The effort **MUST** adapt to relay queue pressure, **MUST NOT** exceed a solution time of ten seconds on a 2020 phone, and **MUST** return to the base effort within ten minutes after the pressure ends.

Rationale. A fixed effort is either useless under attack or hostile in normal operation. The cap keeps a legitimate first contact possible under attack; without it an attacker raises the effort until real users cannot join.

Verification. Test with a simulated flood.

SSP-ABU-007 A relay **MUST** enforce a hard limit of 64 introduction objects per introduction mailbox per 24 hours, which the recipient **MUST NOT** be able to raise above 256.

Rationale. A user does not receive more than a few introductions a day. A ceiling the recipient cannot remove stops an attacker who compromises the client from opening the flood gate.

Verification. Test.

SSP-ABU-008 A client **MUST** let the user rotate the introduction mailbox in one action, which invalidates every contact code issued earlier.

Rationale. The recovery action when a code becomes public. It costs the user the need to reissue codes to people who have not used theirs yet.

Verification. Demonstration.

3.10.3 Relay resource limits

SSP-ABU-009 A relay **MUST** apply a token bucket per link and per source prefix, at /64 granularity for IPv6 and /24 for IPv4.

Rationale. Per-address limits are useless against IPv6, where one host holds a large prefix.

Verification. Test.

SSP-ABU-010 A relay **MUST** limit the mailboxes one link may register, **MUST** state the limit, and **MUST NOT** set it below 512.

Rationale. A normal user needs a few hundred. A lower limit breaks a normal client and no limit lets one link exhaust the address table.

Verification. Test.

SSP-ABU-011 A relay **MUST** require an Equi-X solution for registration when one link exceeds four registrations per minute.

Rationale. Mailbox creation is the cheapest state-creating operation and needs a cost under pressure. The threshold does not affect a normal client, which registers a few mailboxes a day and spreads them under [SSP-MDR-003](#).

Verification. Test.

SSP-ABU-012 A relay **MUST** bound the disk one mailbox uses and the disk all mailboxes use, and **MUST** refuse new registrations as the second bound approaches.

Verification. Test.

SSP-ABU-013 A relay **MUST NOT** let an unauthenticated party cause more than one signature verification and one hash per received frame before that frame passes the length and rate checks.

Rationale. A signature check is the most expensive step, and an attacker must not be able to trigger many of them cheaply.

Verification. Analysis of the relay code path, and a load test.

3.10.4 Sybil resistance

SSP needs none, and that is a design outcome rather than an omission. A Sybil attack pays when identity count buys something: a vote, a reputation score, a share of a resource, or a place in a routing table. An SSP identity gets none of these. It gets no rights at a relay, because a relay does not know identities. It gets no access to a user, because access needs a contact code the user issued. It gets no position in a network structure, because there is no DHT and no federation.

SSP-ABU-014 The protocol **MUST NOT** grant any privilege, priority, or resource share on the basis of the number, age, or connectedness of an identity.

Rationale. Every such grant creates a Sybil incentive and a market for identities.

Verification. Inspection of every operation in [Appendix C](#).

3.10.5 Strategies rejected

Table 48. Abuse strategies considered and rejected.

Strategy	Reason
Reputation scores	A score needs a global view, which needs a central service or a consensus system. A score is also a persistent label on a user: precisely the metadata the protocol works to remove.

Strategy	Reason
Web-of-trust admission	Leaks the social graph to the participants and locks out a new user who knows nobody.
Payment or token deposit	Excludes users with no money or no payment rail, needs a ledger, and makes the network a target for financial regulation.
Phone number verification	A global identifier tying the identity to a state registry and a carrier, and it recreates the enumeration problem.
Central blocklists	A governance function needing an authority, which becomes the target of legal and commercial pressure.
Content filtering at a relay	A relay cannot read content. This is a feature.
CAPTCHA	Needs a vendor, excludes users with a disability, and machine solvers now cost less than the human ones.
Global rate limits per identity	Requires the relay to know the identity.
Adaptive proof of work on all traffic	Costs every honest user battery on every message, and adds nothing in the known-sender case the deposit key already solves.

3.10.6 Residual risk

Attack	Effect and residual risk
A contact turns hostile and floods a mailbox	The quota bounds it and the recipient deletes the mailbox. Residual: the user must act.
An attacker publishes the contact code of a victim	Introductions arrive at the effort cost, capped at 64 a day. Residual: the user must rotate and reissue.
A botnet floods a relay with connections	Token buckets and proof of work on registration. Residual: bandwidth exhaustion remains possible and affects every user of that relay; two relays are required.
An attacker fills a mailbox to block a real message	The quota is per mailbox and only the holder of the deposit secret can write. Residual: a hostile contact can block itself, which is not useful.
A relay operator floods its own users	The relay can forge nothing and can waste storage. Residual: the users move.

Table 49. Residual abuse risk.

3.11 Metadata resistance

3.11.1 Inventory

Table 50. What is exposed, to whom, and what remains.

Value	Seen by	Defence	Residual
Client IP address	Relay, transit	Onion or VPN, by user choice	Full exposure without one choice
Use of SSP	Transit	Binding B with a pluggable transport	Binding A is fingerprintable

Value	Seen by	Defence	Residual
Mailbox address	Relay	Per-pair derivation, daily rotation	Linkable within one day
Mailbox set of one client	Relay	Sharding, spread registration	One relay sees its own shard
Object size	Relay, network	Padding buckets	The bucket index leaks a range
Object count and time	Relay, network	Cover traffic in the high-risk mode	Full exposure by default
Object type class	Relay	None; in the clear	Five values
Deposit key	Relay	Unlinkable to any identity	Links deposits to one sender within an epoch
Blob size	Relay	64 KiB chunks, padded final chunk	Chunk count leaks an approximate size
Identity log	Contacts and serving relays	No name, no external identifier	Device count and change history
Group membership	Nobody outside the group	Pairwise mailboxes	A member sees the membership
Message content	Nobody outside the session	The ratchet	An endpoint sees it

3.11.2 Padding buckets

SSP-MDR-006 Every envelope ciphertext **MUST** be padded to one of the sizes below, and the bucket field **MUST** name that size.

Rationale. Six buckets give a coarse size signal instead of an exact one. Finer would leak more; coarser would waste bandwidth. The 512-byte bucket keeps receipts cheap, and receipts dominate the object count.

Verification. Test.

Index	Size	Typical content
0	512 B	A receipt, a typing notice, a small control object
1	2 KiB	A short text message
2	8 KiB	A long text message, a small group commit
3	32 KiB	A group commit, a blob notice with a large key set
4	64 KiB	The maximum message
5	256 KiB	A large group commit only

Table 51. Padding buckets.

SSP-MDR-007 A client **MUST** pad with random bytes inside the AEAD, never outside it.

Rationale. Padding outside the AEAD is unauthenticated, and an attacker can strip or extend it to signal information.

Verification. Test.

SSP-MDR-008 A client **MUST NOT** choose a bucket larger than the smallest that fits, except in the high-risk mode.

Rationale. Free choice of bucket is a covert channel a compromised client can use to signal to a relay.

Verification. Test.

3.11.3 Deliberate leaks

Three values are visible on purpose.

Leak	Why it exists	What it costs
Object type, five values	The relay needs it for the expiry class and the quota class; without it an introduction and a message share a quota and the abuse model fails.	An observer learns whether an object is a message, an introduction, a commit, a blob notice, or a receipt. Commit traffic therefore reveals that a mailbox belongs to a group member.
Bucket, six values	The relay must check that the length matches, or a sender could signal in the difference.	A size range.
Deposit timing	A store-and-forward relay receives the object at some moment.	Full timing exposure by default.

Table 52. Deliberate plaintext in the envelope.

SSP-MDR-009 No further plaintext field **MUST** be added to an envelope in SSP/1.

Rationale. Each field looks small, and the set is what identifies a user. A version bump is the correct place to add one, with an explicit analysis.

Verification. Inspection.

3.11.4 Sharding and fingerprints

SSP-MDR-010 A client **MUST** distribute its mailboxes across its relay set so that no relay holds more than 60 per cent of them.

Rationale. A relay holding every mailbox of a client sees the full contact count and activity pattern. Sharding bounds the view of one relay; a coalition covering the whole relay set defeats it, and the user chooses the relay set and therefore the coalition risk.

Verification. Analysis of client policy, and Demonstration.

SSP-MDR-011 A client **MUST** use a separate transport connection per relay and **MUST NOT** reuse a source port in a way that correlates them.

Verification. Inspection.

SSP-MDR-012 Every client **MUST** produce a byte-identical HELLO frame for the same version and capability set, and **MUST NOT** include a product name, a version string, a platform value, or a locale.

Rationale. A user agent string turns a client into a distinguisher and can identify one user out of a small population.

Verification. Test with a byte comparison across two implementations.

SSP-MDR-013 A client **MUST NOT** let a user-visible setting change a value the relay sees, beyond the mailbox set and the traffic the user causes.

Rationale. A rare setting combination is a fingerprint.

Verification. Inspection.

SSP-MDR-014 A client **MUST** use the poll interval class for its power state from the fixed set in Appendix B.

Rationale. A custom poll interval identifies a client across relays and across addresses.

Verification. Test.

3.11.5 High-risk mode

SSP-MDR-015 A client **MUST** implement a high-risk mode, switchable per device, with all of: constant-rate traffic of one object every 30 seconds, with padding frames filling empty slots; all traffic over an onion transport using binding B; every envelope padded to at least 8 KiB; and polling only on the fixed schedule, never on a push notification.

Rationale. The mode raises the cost of the T6 attack within one relay view. It costs the user about 23 MiB of upload a day and up to 30 seconds of delivery latency.

Verification. Test with a traffic capture, and analysis of the pattern.

SSP-MDR-016 A client **MUST** state the bandwidth and latency cost before enabling the high-risk mode.

Verification. Demonstration.

3.11.6 Limits

This subsection is normative. An implementation **MUST NOT** claim more than it states.

1. **A global passive adversary defeats the default mode.** An adversary watching both endpoints correlates a deposit at t with a fetch at $t + d$ for small d , and the bucket sizes match. The high-risk mode raises the cost within one relay view and does not defeat an adversary watching both endpoints over a long period.
2. **The relay set is a fingerprint.** A user with an unusual relay set is identifiable by it. Users **SHOULD** choose relays that many users choose, which conflicts with decentralisation. The conflict is stated rather than hidden.
3. **First contact is visible in time.** An introduction deposit tells the relay that some party met the owner of that mailbox. It does not say which party.
4. **Commit traffic is a signal.** A commit object at a mailbox tells the relay that the mailbox belongs to a member of some group. It does not say which group or how large.
5. **A mailbox links a pair within an epoch.** Rotation is daily, and an observer recording everything can chain the epochs of an active pair by activity correlation, without holding the derivation secret.
6. **Nothing protects a user against an endpoint.** TX-02 and TX-03.

3.12 Performance

These are budgets, not aspirations. A design change that breaks one needs a change to this subsection.

Table 53. Performance budgets.

Requirement	Statement	Budget	Method
SSP-PRF-001	Session setup from a cached prekey bundle to first ciphertext, on a 2020 phone	50 ms	Test
SSP-PRF-002	Message encryption for one recipient device, 4 KiB payload	1 ms	Test
SSP-PRF-003	Identity log verification, fifty entries	20 ms	Test
SSP-PRF-004	Group commit object in a group of 4096 devices	256 KiB	Test
SSP-PRF-005	Relay deposit check and quota application, one core	200 μ s	Test
SSP-PRF-006	Registered mailboxes accepted per link	512	Test
SSP-PRF-007	Mobile data per day, 200 contacts and 100 messages	5 MiB	Demonstration
SSP-PRF-008	Local state for 200 contacts and 20 groups, excluding bodies and blobs	50 MiB	Test
SSP-PRF-009	Heap allocations per message in the message path	2	Inspection
SSP-PRF-010	Cold start to first fetch, 2020 phone	2 s	Test
SSP-PRF-011	Equi-X verification at a relay	100 μ s	Test
SSP-PRF-012	Blob encryption and hashing throughput, one core	20 MiB/s	Test

SSP-PRF-013 A client **MUST** batch its fetch operations and **MUST NOT** open one request per mailbox.

Rationale. Two hundred mailboxes at one request each would cost two hundred round trips and a great deal of radio wake time. A batched fetch over one stream costs one.

Verification. Test with a packet capture.

3.13 Usability

Usability is a security property here, because a user who cannot understand a security event cannot act on it.

SSP-USE-002 A client **MUST** present a security event in plain language, and **MUST NOT** make a raw key, hash, or hexadecimal value the primary content of an alert.

Verification. Demonstration.

SSP-USE-003 A client MUST support out-of-band authentication with a 60-bit short authentication string, derived from the SIDs of both parties in a canonical order.

Rationale. Canonical order means both parties see the same words with no role confusion. Sixty bits is enough against an online attack and fits in five words from a 4096-word list.

Verification. Test with the vectors of Appendix E.

SSP-USE-004 A client MUST NOT ask the user to make a cryptographic choice, and MUST NOT offer an option that weakens a default.

Verification. Inspection.

SSP-USE-005 A client MUST tell the user, at the first message to a new contact, that the contact is not yet authenticated out of band.

Verification. Demonstration.

3.14 Interfaces

Interface	Parties	Binding
Relay link	Client and relay	3.8.2 or 3.8.3, frames of 3.5.5
Enrolment	Device and device	Noise IK over a rendezvous mailbox
Contact code	User and user	Bech32m text or QR code
Identity log	Any party	A signed byte string, transport independent
Blob	Client and relay	Chunk group put and get

Table 54. Protocol interfaces.

SSP-IFC-001 A client MUST support the `sunspot:` URI scheme with the forms `sunspot:code/...`, `sunspot:relay/...`, and `sunspot:id/...`, where the last form is display-only.

Rationale. One scheme with a fixed set of forms lets an operating system route a scan or a link. The identifier form is deliberately inert, because a SID alone gives no way to reach a user and [SSP-MDR-005](#) requires that.

Verification. Test.

SSP-IFC-002 A client MUST NOT act on a `sunspot:` URI without an explicit user action.

Rationale. A link in a web page must not be able to add a contact.

Verification. Demonstration.

SSP-IFC-003 An implementation MUST expose the protocol through an interface taking bytes and time and returning bytes and events, and MUST NOT require a network socket inside the protocol core.

Rationale. This is what makes the core testable against vectors and portable to a platform with a different network stack.

Verification. Inspection.

3.15 Design constraints

Table 55. Design constraints.

Requirement	Constraint and reason
SSP-CON-004	The mandatory core MUST be implementable with no dynamic allocation in the parsing path, for an embedded client and for a relay under load.
SSP-CON-005	The protocol MUST NOT need a wall clock accurate to better than five minutes. A protocol that needs an accurate clock needs a time source, and a time source is a trusted party.
SSP-CON-006	The protocol MUST NOT need a stable IP address, a public port, or an inbound connection at a client. Most clients sit behind address translation.
SSP-CON-007	The protocol MUST work with a relay having one core, 2 GiB of memory, and 100 GiB of disk, serving 10 000 users. A relay must be cheap to run or there will be few relays.
SSP-CON-008	A relay MUST NOT need state surviving a restart beyond the mailbox table and the object store.
SSP-CON-009	No protocol operation MAY need more than four round trips.
SSP-CON-010	The protocol MUST NOT contain a licence-encumbered algorithm or a patented mechanism. Every implementation must be free to write.

3.16 Quality attributes

3.16.1 Security

Property	Status	Source
Confidentiality of content	yes	3.7, 3.9.1
Integrity and authenticity	yes	AEAD and the ratchet
Forward secrecy	yes, per message	Ratchet plus key deletion
Post-compromise security	yes, after one round trip	The DH ratchet step
Deniability	yes, offline	No signature on messages
Post-quantum confidentiality	yes, for a recorded transcript	Hybrid ML-KEM-768
Post-quantum authenticity	no, in SSP/1	3.7.12
Identity binding	to a log, not a name	3.6
Transparency of key change	detectable by a contact	3.6.11
Replay resistance	yes, at ratchet and relay	3.9.2, 3.5.6
Downgrade resistance	yes, transcript binding	3.17.2

Table 56. Security properties.

Three places carry most of the implementation risk. The *parser* reads attacker bytes; 3.5.4 fixes the grammar, 5.3 forbids allocation and requires a bounds check per field, and a fuzz campaign

is mandatory. The *state machine* holds skipped keys and epochs; hard limits on both stop an attacker forcing unbounded state. The *key store* separates keys by purpose and encrypts at rest with crypto-erase.

SSP-SEC-002 An implementation MUST NOT log a key, a plaintext, a mailbox address, or a SID at any log level.

Verification. Inspection, plus a pattern-matching test in the build.

SSP-SEC-003 An implementation MUST perform cryptographic comparisons in constant time.

Verification. Inspection and Analysis.

SSP-SEC-004 An implementation MUST NOT continue after a cryptographic check fails, and MUST NOT reveal the reason to the peer beyond the defined error codes.

Rationale. A detailed error is an oracle.

Verification. Test.

3.16.2 Privacy

SSP-PRV-001 A client MUST NOT send telemetry, a crash report, or a usage statistic carrying a mailbox address, a SID, a relay set, a contact count, or a group count.

Verification. Inspection.

SSP-PRV-002 A client MUST NOT enable a push notification service that reveals a message or a sender to a third party, and MUST use a content-free wake signal if it uses a platform push service at all.

Rationale. A platform push service sits outside the trust model. A wake signal with no content tells it only that a device has traffic, which the network layer already reveals.

Verification. Test with a capture of the push payload.

SSP-PRV-003 A client MUST make local storage of a conversation removable without a trace in an index, a search database, or a thumbnail cache.

Verification. Demonstration.

SSP-PRV-004 The protocol MUST NOT define a mechanism by which one party learns whether another is online, unless that party sends a message.

Rationale. Presence is high-resolution activity metadata, and it is the value users leak most freely. A presence service would also require a relay to know an identity.

Verification. Inspection of the operation registry.

3.16.3 Reliability, maintainability, portability

SSP-SEC-005 A client **MUST** recover from a corrupt local store by re-derivation from the event log, and **MUST NOT** lose a key it cannot re-derive.

Verification. Test with a truncated store file.

SSP-SEC-006 A relay **MUST** survive power loss with no loss of an acknowledged deposit, and **MUST** flush an object to disk before acknowledging it.

Verification. Test with power-loss injection.

SSP-GEN-008 Every wire object **MUST** have a test vector, and a change to an object **MUST** come with a change to the vector.

Verification. Inspection during review.

SSP-GEN-009 The protocol core **MUST** be separable from transport, storage, and user interface at build level.

Verification. Inspection of the crate graph.

SSP-GEN-010 The protocol core **MUST** build for a 32-bit target with no operating system, given an entropy source and a clock.

Rationale. A hardware token, a router, and a low-cost device are all valid clients. A protocol needing a 64-bit host with an allocator excludes them for no reason.

Verification. Test with a bare-metal build.

3.17 Extensibility, versioning, and compatibility

3.17.1 Extension points

There are three, and no more: the content type of a message body, the extension list of a message, and the identity log entry type range 0x80 to 0xFF.

SSP-EXT-001 An extension **MUST NOT** change a cryptographic mechanism, a key schedule, a mailbox derivation, or a frame format.

Rationale. An extension touching the cryptography is a new protocol version under a false name. This rule is what stops SSP/1 becoming a family of incompatible dialects.

Verification. Inspection at registry review.

SSP-EXT-002 A receiver **MUST** ignore an unknown non-critical extension and **MUST** reject the whole object on an unknown critical extension.

Rationale. The rule TLS and X.509 use. It lets a sender state whether the extension is needed for correct handling.

Verification. Test.

SSP-EXT-003 A receiver **MUST** include an unknown extension in the bytes it hashes for the message identifier, and **MUST NOT** strip it before re-encoding.

Rationale. Stripping breaks the content address and the signature.

Verification. Test.

SSP-EXT-004 A code point MUST come from the registry of Appendix C, and an experiment MUST use a private-use range.

Rationale. Two implementations choosing the same number for different meanings break the network in a way that is hard to diagnose.

Verification. Inspection.

Registry policy: 0x0000–0x7FFF assigned by the maintainers with a specification; 0x8000–0xEFFF first come, first served with a public description; 0xF000–0xFFFF private use, never on a public network.

Fixed for the life of SSP/1. The primitive set, the frame header, the envelope layout, the mailbox derivation, the SAKA construction, the ratchet parameters, the group ciphersuite, and the padding bucket set. A change to any of them needs SSP/2.

3.17.2 Version negotiation

HELLO carries the supported versions, a capability bitmap, and a client nonce. HELLO_ACK carries the selection, the relay capability bitmap, the policy block, a relay nonce, and a transcript MAC:

$$T = H(\text{SSP1/hello} \parallel \text{HELLO} \parallel \text{HELLO_ACK without the MAC}), \quad mac = KH(k_{\text{link}}, T)$$

SSP-EXT-005 A relay MUST bind the negotiated version and capability set to the transport key material with that MAC, and a client MUST check it.

Rationale. Downgrade protection. An attacker editing HELLO in flight cannot produce a valid MAC, because it does not hold the transport secret.

Verification. Test with a modified HELLO.

SSP-EXT-006 A client MUST NOT offer a version below its highest supported version to avoid a failure, and MUST NOT retry at a lower version after a failure.

Rationale. An automatic downgrade retry is a downgrade oracle, and it defeats the MAC above at the application level. Several deployed protocols lost their downgrade protection exactly this way.

Verification. Test.

SSP-EXT-007 The end-to-end version MUST be negotiated inside the SAKA transcript, and MUST NOT be taken from the relay link.

Rationale. The relay is not trusted and cannot take part in an end-to-end negotiation.

Verification. Test.

A capability is a bit in a 128-bit map: it enables an optional behaviour and never changes a mandatory one. Bit 0 blob storage, bit 1 push wake, bit 2 datagram padding, bit 3 extended object lifetime, bit 4 proof of work currently not required, bits 5 to 127 reserved.

SSP-EXT-008 A party MUST NOT use a capability the other party did not offer, and MUST treat an unknown bit as not offered.

Verification. Test.

3.17.3 Compatibility

SSP-EXT-009 A minor revision of SSP/1 **MUST** be additive, **MUST NOT** change an existing field, and **MUST NOT** change the meaning of an existing code point.

Verification. Inspection at review.

SSP-EXT-010 An SSP/1 client **MUST** interoperate with any other SSP/1 client for every mandatory function, whatever the minor revision.

Verification. The interoperability suite of 4.5.

The one break with the past. SSP/1 is not compatible with earlier SunSpot builds. The break happens once, and the reason is recorded here so that habit cannot repeat it: the old format has no canonical encoding, so no signature over it is sound; the old identity model has no key rotation, so every old identity must be reissued anyway; and the old addressing model uses a stable per-user identifier at the relay, which is exactly the property 3.8.6 removes, so a bridge would reintroduce the leak for every bridged conversation.

SSP-EXT-011 An SSP/1 client **MUST NOT** implement a bridge, a gateway, or a translation layer to a pre-SSP/1 build.

Rationale. A bridge would move plaintext, or the metadata leak, into the new network. Migration is a user action: create a new identity and send a contact code over the old system.

Verification. Inspection.

SSP-EXT-012 SSP/2 **MUST** run on a separate ALPN value and a separate frame version, and a relay **MUST** be able to serve both at once on one port.

Rationale. A flag day across a decentralised network is not possible. Two versions in parallel at one relay is the only way a network can move.

Verification. Demonstration.

4 Verification

This section mirrors Section 3, as ISO/IEC/IEEE 29148 requires, and states how a reviewer confirms that an implementation meets each requirement.

4.1 Strategy

SSP-GEN-011 An implementation **MUST** pass every V1 and V2 case before any party calls it conformant.

Rationale. Encoding and primitives are where silent divergence starts, and a divergence there breaks every higher property.

Verification. Test.

Level	Object	Method
V1	A primitive and its parameters	Test against published vectors
V2	An object encoding	Test against the SSP vectors
V3	A state machine	Test against a scripted trace, plus model checking
V4	A protocol exchange	Demonstration between two implementations
V5	A property	Analysis, with a proof or a symbolic model
V6	A limit or a policy	Inspection of code and document

Table 57. Verification levels.

4.2 Selected cases

Table 58. Representative verification cases.

Requirement	Method	Case
SSP-IDN-001	Test	The SID of the reference genesis entry matches the published value.
SSP-IDN-004	Test	A log with a broken link at entry seven is rejected whole.
SSP-IDN-006	Test	A log with sequence 0, 1, 3 is rejected.
SSP-IDN-011	Test	A relay serving a fork triggers the alert path and stops sends.
SSP-DEV-002	Test	A log with nine active devices is rejected.
SSP-DEV-004	Test	The enrolment handshake matches the Noise vector; a wrong pre-shared key fails.
SSP-KEY-009	Test	A recovery claim with a 71-hour window is rejected.
SSP-KEY-010	Test	A veto inside the window voids the claim.
SSP-CRY-002	Test	Every hash output matches the BLAKE3 vector set.
SSP-CRY-003	Test	Each small-order X25519 input aborts the operation.
SSP-CRY-007	Test	The full ZIP 215 vector set gives the expected accept and reject sets.
SSP-CRY-009	Test	Removing either the X25519 or the ML-KEM output changes the session secret.
SSP-CRY-013	Test	Key generation blocks until the entropy source is ready.

4.3 Parser verification

SSP-GEN-012 An implementation **MUST** run a coverage-guided fuzz campaign of at least 10^9 executions against the envelope, message, identity log, and frame parsers, with no crash, hang, or memory error.

Rationale. The parser reads attacker bytes and is the highest-value target.

Verification. Test, with the campaign log as evidence.

SSP-GEN-013 An implementation **MUST** include a differential test against a second implementation for the accept or reject decision of every parser.

Rationale. Two parsers differing on one byte string give an attacker a way to split the network.

Verification. Test.

4.4 Property verification

Property	Method	Evidence
Session confidentiality and authenticity	Analysis	A symbolic model of SAKA and the ratchet in ProVerif or Tamarin
Downgrade resistance	Analysis, Test	Transcript binding inside the model
Group forward silence after removal	Test	A removed member cannot decrypt the next epoch, over 1000 random membership sequences
Convergence of concurrent commits	Test	10^5 random concurrency traces reach one epoch state on every member
Convergence of display order	Test	10^5 random delivery orders of one graph give one order
Mailbox unlinkability	Analysis	The derivation is a PRF, so two epochs are indistinguishable without the root

Table 59. Property verification.

4.5 Interoperability suite

SSP-GEN-014 The maintainers **MUST** publish an interoperability suite holding a reference relay, a scripted client, a hostile relay, and a traffic recorder.

Rationale. G-01 cannot be verified by reading. It needs two implementations and a harness a third party can run.

Verification. Demonstration.

The hostile relay must be able to drop an object silently; reorder objects and reuse a sequence number; serve a stale or forked identity log; serve one prekey to two requesters; lie about its policy limits; delay an object arbitrarily; and replay a deposit. A client is tested against all seven.

4.6 Traceability

Appendix A links each requirement to a goal, a threat, and a verification method. A requirement tracing to neither a goal nor a threat is unnecessary and **MUST** be removed.

5 Implementation guidance

This section is informative. It records the implications of the design for a Rust implementation, because several rules above exist for reasons that only become visible at the code level.

5.1 Shape

The protocol core is a state machine taking bytes and time and returning bytes and events. It owns no socket, no thread, no timer, and no file, which is what makes every verification case in Section 4 a table of inputs and expected outputs with no network and no clock.

Crate	Contents
ssp-wire	Encoding and decoding, zero copy, no allocation in the parse path
ssp-crypto	Thin traits over the primitives, no algorithm choice
ssp-identity	Identity log, entries, verification, fork detection
ssp-session	SAKA and the double ratchet
ssp-group	TreeKEM, epochs, commit resolution
ssp-mailbox	Derivation, envelope construction, padding
ssp-relay-proto	Frames, operations, policy; no input or output
ssp-transport	QUIC and TCP-with-Noise bindings; async
ssp-store	Local encrypted store and event log; platform-specific
ssp-client	Orchestration
ssp-relay	The relay server
ssp-testkit	Vectors, hostile relay, fuzz targets

Table 60. Crate layout.

5.2 Crate boundaries

The rule that keeps this honest: the first six crates must not depend on an async runtime, on `std::net`, or on anything that opens a file. A dependency check in continuous integration enforces it.

5.3 Zero-copy parsing

The encoding is designed for a borrowed parse: a parsed object holds slices into the input buffer and copies nothing.

```
pub struct EnvelopeRef<'a> {
    pub etype:      u8,
    pub mailbox:    &'a [u8; 32],
    pub nonce:      &'a [u8; 16],
    pub bucket:     u8,
    pub ciphertext: &'a [u8],
    pub signature:  &'a [u8; 64],
}
// parse(): check total length against the bucket table first; take fixed
// fields by exact slice, never by index arithmetic; reject trailing bytes.
```

Three habits matter. Use checked splitting, never an index that a wire-controlled length field computes. Return a borrowed view and let the caller decide to own it, which is what keeps [SSP-PRF-009](#) achievable. Make the parse function total: it takes bytes, returns a result, and panics on nothing.

Do not put `serde` on the wire. The protocol reason is in [3.5.4](#); the code reason is that a derive macro places the wire format under the control of a struct definition, so a field reorder during a refactor silently changes the network. `serde` is fine for a configuration file and a local store, where the format carries no cross-implementation meaning.

5.4 Type discipline

The protocol carries many 32-byte values with very different meanings. A raw `[u8; 32]` everywhere makes it possible to pass a mailbox address where a SID belongs and the compiler will not object. Wrap each in a distinct newtype with no dereference to the inner array. Apply

the same discipline to keys: a signing key should not implement `Clone`, and a secret type should implement neither `Debug` nor `Display`, which turns [SSP-SEC-002](#) into a compile-time property rather than a review item.

5.5 Zeroization

Rust makes [SSP-KEY-003](#) harder than it looks. A growable buffer reallocates and leaves the old allocation holding the key. A clone of a secret makes a copy no destructor tracks.

- Hold secrets in a fixed-size array inside a wrapper whose destructor performs a volatile write. Use an established zeroization crate rather than a hand-written loop the optimiser may remove.
- Never place a secret in a growable container.
- Do not derive `Clone` on a secret type. Where a copy is genuinely needed, make it an explicit method with a name a reviewer notices.
- Lock the pages of the key store where the platform allows it, so a secret does not reach a swap file.
- Document the truth: a virtual machine that snapshots memory, and a core dump, both defeat this. Zeroization narrows the window and does not close it.

5.6 Async design

The transport layer is async and the core is not. The boundary is one task per relay connection with one channel into the core.

- Give the core exclusive ownership of protocol state behind one actor task. Shared mutable state behind a lock invites a deadlock between a fetch task and a send task.
- Put a deadline on every operation, and make the cancel path leave the ratchet whole. Ratchet state must advance atomically with the durable write, or a crash between the two loses a key.
- Use one connection per relay with one stream per logical operation, and do not spawn a task per mailbox.

5.7 Constant time

Use an established constant-time comparison crate for any secret-derived value. Comparing a MAC tag with slice equality is a timing oracle, and it is the defect reviewers find most often. The parser is not constant time and does not need to be, because it reads public bytes; do not confuse the two cases and do not slow the parser for nothing.

5.8 Build order

Each stage below is testable before the next begins, and every stage before the fifth is a pure function of bytes, so a defect is found by a vector rather than a packet capture.

1. `ssp-wire`, against the encoding vectors. No network.
2. `ssp-identity`, against the log vectors and the fork cases.
3. `ssp-session`, against the SAKA and ratchet vectors.
4. `ssp-mailbox`, against the derivation vectors.
5. `ssp-relay-proto` and a relay with an in-memory store.

6. `ssp-transport`, binding A first, binding B second.
7. `ssp-group`, against the concurrency traces.
8. Blobs, then the self group, then the high-risk mode.

6 Research agenda

Each item below has a decision in SSP/1. The entry states what would have to change for that decision to be revisited.

6.1 Private retrieval

Now. A client tells a relay which mailbox addresses it reads, and sharding bounds the view of any one relay.

What would change it. A practical private information retrieval scheme letting a client fetch from a set of N mailboxes without naming them. Current schemes cost either linear server work per query or large client state, and neither is acceptable at the scale of 3.12. A single-server scheme with sublinear amortised cost and a small hint would remove the strongest remaining relay-side leak.

6.2 Group fan-out

Now. One envelope per recipient device, buying full membership confidentiality against a relay.

What would change it. A construction letting a relay serve one ciphertext to N subscribers without learning that those N form a set. Broadcast encryption solves the key half and not the addressing half. An oblivious relay inside a trusted execution environment would work and would add a hardware vendor to the trust model, which the trust assumptions do not accept.

6.3 Post-quantum authenticity at an acceptable size

Now. Ed25519 with a defined migration.

What would change it. A post-quantum signature with a public key under 200 bytes and a signature under 1 KiB, with a comparable analysis record. Current lattice and hash-based schemes miss this by a large factor, and the identity log is where the size hurts most.

6.4 Anonymous credentials for first contact

Now. A contact code plus proof of work.

What would change it. A blind-signature or oblivious-PRF token that a recipient issues, a sender redeems once, and a relay verifies without linking redemption to issue. That would replace proof of work with an unlinkable capability and remove the battery cost of the introduction path. The construction is well understood; the open problem is distributing tokens to a party the recipient has never met, which is exactly the case the contact code covers.

6.5 Traffic analysis at an acceptable cost

Now. An opt-in constant-rate mode, with the default mode explicitly not resistant to a global passive adversary.

What would change it. A mix network design a phone can afford, with latency a messaging user accepts. The Loopix family is closest, and its cost model assumes a persistently connected client. A version surviving a mobile radio duty cycle would let the mode become the default.

6.6 Formal verification of the composition

Now. A symbolic model of SAKA and the ratchet.

What would change it. A machine-checked proof covering the identity log, the session layer, the group layer, and the addressing layer in one model. The parts have proofs; the composition does not, and the composition is where a protocol usually fails.

A Traceability matrix

A requirement tracing to neither a goal nor a threat is unnecessary.

Table 61. Requirement traceability.

Requirement	Goal	Threat	Method
SSP-GEN-001	G-01	—	Inspection, Test
SSP-GEN-002	G-07	—	Inspection
SSP-GEN-003	G-02	TH-01	Test
SSP-GEN-004	G-01	—	Demonstration
SSP-GEN-005	G-12	—	Inspection
SSP-GEN-006	G-04	TH-11	Inspection, Test
SSP-GEN-007	G-02	TH-04	Test
SSP-GEN-008	G-01, G-09	—	Inspection
SSP-GEN-009	G-12	—	Inspection
SSP-GEN-010	G-12	—	Test
SSP-GEN-011	G-01	—	Test
SSP-GEN-012	G-12	TH-19	Test
SSP-GEN-013	G-01	TH-19	Test
SSP-GEN-014	G-01	—	Demonstration
SSP-IDN-001 to 003	G-05	TH-04	Test, Inspection
SSP-IDN-004 to 006	G-05	TH-04	Test
SSP-IDN-007	G-09	—	Test
SSP-IDN-008, 009	G-05	TH-04	Test
SSP-IDN-010 to 013	G-05	TH-09	Test, Demonstration
SSP-IDN-014 to 017	G-04, G-08	TH-10, TH-13	Test, Demonstration
SSP-DEV-001 to 003	G-06	TH-05	Inspection, Test
SSP-DEV-004 to 007	G-06	TH-05	Test, Demonstration
SSP-DEV-008 to 011	G-03, G-06	TH-18	Test
SSP-KEY-001 to 003	G-02, G-03	TH-01, TH-02	Inspection, Test
SSP-KEY-004 to 006	G-03, G-05	TH-02	Test

Requirement	Goal	Threat	Method
SSP-KEY-007 to 012	G-05	TH-06	Test, Demonstration
SSP-CRY-001 to 008	G-02, G-12	TH-01, TH-20	Test, Analysis
SSP-CRY-009 to 011	G-10	TH-03	Test, Analysis
SSP-CRY-012 to 014	G-02	TH-01	Test, Inspection
SSP-NET-001, 002	G-11	TH-15	Test
SSP-NET-003	G-07	TH-04	Test
SSP-NET-004, 005	G-07	TH-08	Test, Inspection
SSP-NET-006, 007	G-02	TH-20, TH-07	Test
SSP-NET-008 to 011	G-07	TH-15	Test, Demonstration
SSP-NET-012, 013	G-07	TH-10	Demonstration, Analysis
SSP-RLY-001 to 004	G-04, G-08	TH-14, TH-15	Test
SSP-RLY-005 to 007	G-11	TH-08	Test
SSP-RLY-008, 009	G-04	TH-11	Inspection
SSP-RLY-010 to 013	G-11	TH-15	Test
SSP-RLY-014	G-07	—	Demonstration
SSP-MSG-001 to 003	G-08	TH-07, TH-14	Test
SSP-MSG-004 to 008	G-02, G-03	TH-01, TH-04	Test, Analysis
SSP-MSG-009 to 012	G-03	TH-02	Test
SSP-MSG-013 to 015	G-02	TH-04	Test
SSP-MSG-016 to 020	G-01	TH-08	Test, Demonstration
SSP-MSG-021 to 023	G-04	TH-11	Test, Inspection
SSP-GRP-001 to 006	G-02, G-03	TH-18	Test, Inspection
SSP-GRP-007	G-04	TH-17	Test, Analysis
SSP-GRP-008 to 010	G-01	TH-08	Test
SSP-GRP-011, 012	G-13	—	Test, Demonstration
SSP-ATT-001 to 006	G-02, G-04	TH-12	Test
SSP-SYN-001 to 006	G-06	TH-05	Test, Demonstration
SSP-STO-001 to 005	G-03	TH-02	Inspection, Demonstration
SSP-ABU-001 to 004	G-08	TH-14	Test
SSP-ABU-005 to 008	G-08	TH-13	Test, Demonstration
SSP-ABU-009 to 013	G-08, G-11	TH-15	Test, Analysis
SSP-ABU-014	G-08	TH-16	Inspection
SSP-MDR-001 to 005	G-04	TH-10, TH-11	Test, Analysis

Requirement	Goal	Threat	Method
SSP-MDR-006 to 009	G-04	TH-12	Test, Inspection
SSP-MDR-010 to 014	G-04	TH-12	Test, Analysis
SSP-MDR-015, 016	G-04, G-13	TX-01	Test, Demonstration
SSP-SEC-001 to 006	G-02	TH-01, TH-19	Inspection, Test
SSP-PRV-001 to 004	G-04	TH-11	Inspection, Test
SSP-PRF-001 to 013	G-11, G-12	—	Test, Demonstration
SSP-USE-001 to 005	G-13	TH-04	Demonstration
SSP-IFC-001 to 003	G-01	TH-10	Test, Inspection
SSP-CON-001 to 010	G-09, G-12	TH-19	Test, Inspection
SSP-EXT-001 to 012	G-09	TH-20	Test, Inspection

B Constants and registries

B.1 Sizes

Constant	Value	Constant	Value
SID	32 B	Maximum envelope	262 144 B
Mailbox address	32 B	Maximum blob	104 857 600 B
Device identifier	8 B	Maximum identity log	256 KiB, 1024 entries
Deposit nonce	16 B	Devices per identity	8
Ed25519 signature	64 B	Identities per group	1024
X25519 public key	32 B	Devices per group	4096
ML-KEM-768 key	1184 B	Parents per message	8
ML-KEM-768 ct	1088 B	Relays per relay set	4
Blob chunk	65 536 B	Mailboxes per link, min	512

Table 62. Size constants.

B.2 Times

B.3 Argon2id parameters

Recovery parameters are heavier deliberately: recovery happens once, in a crisis, and protects a key an attacker can grind offline for years. A store derivation happens at every unlock and must not annoy the user into a shorter passphrase.

B.4 Derivation labels

B.5 Poll interval classes

Constant	Value	Constant	Value
Mailbox epoch	24 h	Old epoch retention in a group	7 days
Mailbox overlap window	24 h	Skipped key retention	7 days
Recovery delay window	72 h	Message queue lifetime	7 days
Signed prekey rotation	7 days	Default object expiry	14 days
Link and KEM rotation	180 days	Group commit expiry	30 days
Root key rotation	365 days	Clock skew tolerance	5 min
Group update proposal	7 days	Future timestamp rejection	24 h
Group commit interval	30 days		

Table 63. Time constants.

Use	Memory	Iterations	Parallelism	Output
Recovery key derivation	256 MiB	4	2	32 B
Local store key from a passphrase	64 MiB	3	4	32 B

Table 64. Argon2id parameters.

Label	Parent key	Output
SSP1/saka-root	concatenated DH and KEM secrets	64 B
SSP1/saka-transcript	plain hash	32 B
SSP1/hybrid-combine	plain hash	64 B
SSP1/ratchet-root	root key	64 B
SSP1/ratchet-chain	chain key	64 B
SSP1/ratchet-header	chain key	32 B
SSP1/mailbox-root	session secret	32 B
SSP1/mailbox-addr	mailbox root	32 B
SSP1/deposit-seed	mailbox root	32 B
SSP1/blob-key	random	32 B
SSP1/store-key	device master key	32 B
SSP1/channel-epoch	previous epoch key	32 B
SSP1/commit-tiebreak	plain hash	32 B
SSP1/pow	plain hash	32 B
SSP1/rendezvous	plain hash	32 B
SSP1/recovery-salt	plain hash	16 B
SSP1/sas	session secret	8 B
SSP1/deposit	signature domain	—
SSP1/mbreg	signature domain	—
SSP1/idlog	signature domain	—
SSP1/msg	hash domain	—
SSP1/hello	transcript domain	—

Table 65. Key derivation label registry.

Class	Power state	Interval
0	Foreground, mains power	Persistent subscription
1	Foreground, battery	Persistent subscription
2	Background, mains power	60 s
3	Background, battery	300 s
4	Low power	900 s
5	High-risk mode	30 s, fixed, with cover traffic

Table 66. Poll interval classes. A client uses one of these values and no other, so that the poll pattern is not a fingerprint.

C Code point registries

C.1 Error codes

Code	Name	Meaning
0x0001	PROTOCOL_ERROR	A frame broke the grammar.
0x0002	VERSION_UNSUPPORTED	No common version.
0x0003	MAILBOX_UNKNOWN	No registration for that address.
0x0004	MAILBOX_FULL	The byte limit is reached.
0x0005	QUOTA_EXCEEDED	The object count limit is reached.
0x0006	BAD_SIGNATURE	A signature did not verify.
0x0007	REPLAY	The deposit nonce is in the cache.
0x0008	POW_REQUIRED	A proof of work is needed.
0x0009	POW_INVALID	The solution failed.
0x000A	RATE_LIMITED	A token bucket is empty.
0x000B	TOO_LARGE	A length limit is exceeded.
0x000C	RELAY_FULL	The relay accepts no new mailboxes.
0x000D	NOT_SUPPORTED	The capability was not offered.
0x000E	INTERNAL	An unspecified relay failure.

Table 67. Error codes. A relay **MUST NOT** define a code that reveals a property of a mailbox or of another client.

C.2 Message content types

D Object layouts

D.1 Mailbox registration

D.2 Identity log entry, device addition

Fields that do not align to a 32-bit boundary are given as offsets and sizes. Every value is big-endian and every offset is in bytes from the start of the entry.

D.3 Message header

Code	Name	Code	Name
0x0001	Text, UTF-8	0x0100	Outgoing message mirror
0x0002	Delivery receipt	0x0101	Read state update
0x0003	Read receipt	0x0102	Contact list change
0x0004	Typing notice	0x0103	Mailbox state share
0x0005	Blob reference	0x0104	Profile change
0x0006	Reaction	0x0105	Protocol-affecting setting
0x0007	Edit	0x0106	History bundle chunk
0x0008	Delete request	0x0200	Group proposal
0x0009	Reply marker	0x0201	Group commit
0x000A	Contact code share	0x0202	Group welcome
0x000B	Log head gossip	0x0203	Group metadata change
0x000C	Group welcome pointer	0x0204	Channel epoch key
0x0010– 0x001F	Reserved for media		

Table 68. Content types. Conversation types on the left, synchronisation and group control on the right.

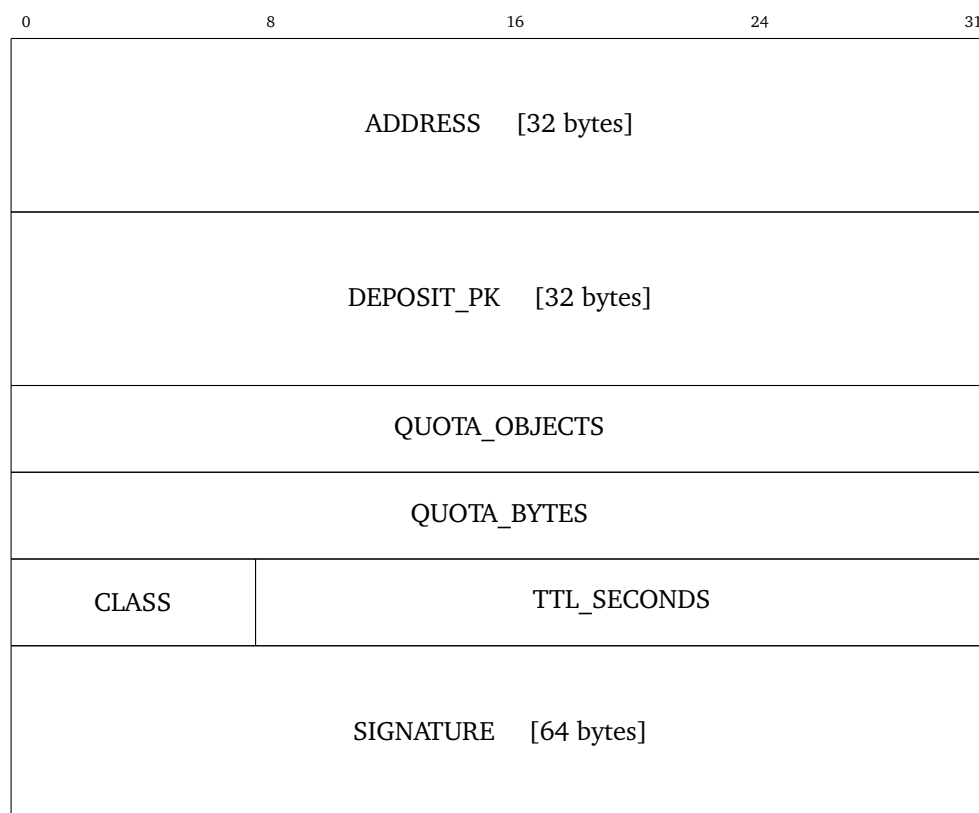


Figure 16. Mailbox registration. The signature is Ed25519 by the deposit secret over SSP1/mbreg, a zero byte, and every field above it.

Offset	Size	Field	Value
0	12	LABEL	SSP1/idlog and two zero bytes
12	1	VERSION	0x01
13	1	TYPE	0x02, device addition
14	4	SEQ	One more than the previous entry
18	32	PREV	Hash of the previous entry
50	8	TIME	Milliseconds since the UNIX epoch
58	8	NOT_BEFORE	When the entry takes effect
66	8	DEVICE_ID	First eight bytes of $H(\text{DEVICE_PK})$
74	32	DEVICE_PK	Ed25519 signature key
106	32	LINK_PK	X25519 static key
138	1184	KEM_PK	ML-KEM-768 encapsulation key
1322	4	CAPS	Capability bits, including the enrol capability
1326	8	ADDED_AT	Timestamp
1334	2	SIG_COUNT	Number of signatures that follow
1336	$72n$	SIGNATURES	n pairs of KEY_ID [8] and SIG [64]

Table 69. Device addition entry. Total size with one signature: 1408 bytes.

Offset	Size	Field	Value
0	12	LABEL	SSP1/msg and four zero bytes
12	1	VERSION	0x01
13	32	SENDER_SID	The identifier of the sending identity
45	8	SENDER_DEV	The identifier of the sending device
53	32	CONV_ID	Pair hash, or group identifier
85	32	MSG_ID	Content address, zero while hashing
117	8	COUNTER	Per sender device, per conversation
125	8	SENT_AT	Advisory sender clock
133	2	PARENT_COUNT	1 to 8
135	$32p$	PARENTS	p parent message identifiers
—	32	LOG_HEAD	Identity log head of the sender
—	2	FLAGS	Reserved, zero in SSP/1
—	2	CONTENT_TYPE	From the content type registry
—	4	PAYLOAD_LENGTH	Byte count of the payload
—	var	PAYLOAD	Body, per content type
—	2	EXT_COUNT	Number of extensions
—	var	EXTENSIONS	Triples of type, length, and value

Table 70. Message header and body. Offsets after the parent list depend on the parent count, so they are given as sizes only. Minimum header size, with one parent and no extensions: 209 bytes.

E Test vectors

Publishing a fixed set of vectors gives an implementer a small number of cases; publishing a deterministic generator gives an unlimited set. SSP does both.

Every value derives from one seed and a path:

$$\begin{aligned} \text{value}(\text{path}, n) &= \text{BLAKE3}_{\text{keyed}}(\text{seed}, \text{SSP1/vector}/\|\text{path}\|)[0..n] \\ \text{seed} &= H(\text{sunspot-proposal-a-vector-seed-v1}) \end{aligned}$$

A key pair uses $\text{value}(\text{path}, 32)$ as the seed of the relevant key generation function. The generator therefore needs no random source, and two implementations produce the same corpus byte for byte.

Path prefix	Content
identity/	100 identity logs with rotation, device change, and recovery, plus 200 invalid variants
enrolment/	50 enrolment transcripts and 50 failure cases
wire/	500 valid encodings and 2000 malformed or non-canonical variants, each with its expected rejection reason
labels/	Every derivation label with a known input and output
recovery/	Argon2id derivations for 20 passphrases
crypto/	Every ZIP 215 case and the small-order X25519 set
mailbox/	1000 mailbox derivations across 30 epochs
ordering/	100 message graphs, each with 100 random delivery orders and one expected display order
group/	100 group traces with concurrent commits and the expected converged state
sas/	100 short authentication string derivations

Table 71. Vector corpus.

SSP-GEN-015 An implementation **MUST** reproduce the full corpus from the seed and match the published digest of the corpus.

Rationale. One digest makes the corpus verifiable in a single comparison and makes silent divergence impossible to miss.

Verification. Test.

Each invalid case carries the bytes, the expected error code, and the clause it tests. An implementation rejects the bytes and returns that code, which is what makes two parsers agree.

F Comparison with existing systems

Table 72. SSP against comparable systems.

Property	Signal	Matrix	SimpleX	Session	SSP
Identifier	phone number	user on a home-server	none, per queue	public key	hash of a log genesis

Property	Signal	Matrix	SimpleX	Session	SSP
Stable across key change	yes	yes	n/a	no	yes
Central service needed	yes	per home-server	no	node network	no
Directory or lookup	yes, enclave	yes	no	no	no
Relay learns the recipient	yes	yes	no	partly	no
Address changes over time	no	no	per queue	no	daily
Multi-device	with a primary	yes	limited	limited	no primary
Group protocol	sender keys	Megolm	pairwise queues	closed groups	TreeKEM, no ordering service
Membership hidden from the server	no	no	yes	partly	yes
Post-quantum key agreement	yes	no	yes	no	yes, hybrid
Spam defence	phone number	server policy	out-of-band links	proof of work	recipient capability, PoW at first contact only
Value of seizing a server	some metadata	room state	little	little	ciphertext and opaque addresses
Federation	no	yes	no	n/a	no, by decision
Needs the reference client to interoperate	in practice	no	no	in practice	no, by requirement

Three differences deserve a note.

Against Signal. Signal is the reference for cryptographic quality and SSP reuses its ratchet. The difference is identity and infrastructure. Signal ties an account to a phone number and runs one service, which gives it a very good spam answer and a very good user experience, and gives a central point a state can pressure. SSP trades the user experience for the removal of that point.

Against SimpleX. SimpleX reaches a similar addressing property with unidirectional queues and no long-term identity at all. SSP keeps a long-term identity, because a stable identifier is what makes rotation, recovery, and a durable contact list possible, and then hides that identity from the relay with the pairwise derivation. The aim is to hold both properties at once.

Against Matrix. Matrix is the reference for an open specification with many implementations, and SSP copies that discipline. It rejects the replication model: sending room state to every participating server multiplies metadata exposure by the server count, and the homeserver holds the account.

G Acronyms

Term	Expansion	Term	Expansion
AEAD	authenticated encryption with associated data	PCS	post-compromise security
ALPN	application-layer protocol negotiation	PIR	private information retrieval
DAG	directed acyclic graph	PoW	proof of work
DH	Diffie-Hellman	PSK	pre-shared key
DHT	distributed hash table	QUIC	the transport of RFC 9000
KDF	key derivation function	RPK	raw public key
KEM	key encapsulation mechanism	SAKA	SunSpot asynchronous key agreement
MAC	message authentication code	SAS	short authentication string
MLS	messaging layer security	SID	SunSpot identifier
STE	Simplified Technical English	SSP	SunSpot protocol
SWE	SunSpot wire encoding	TTL	time to live
VOPRF	verifiable oblivious pseudorandom function	XOF	extendable output function

Table 73. Acronyms.